
华中科技大学创新创业训练计划

23 级项目研究总结报告（2025 年度）

项目名称 微环调制器波长调节驱动器设计

负责人 张祥云

所在院系 集成电路学院

项目组成员 庾宇、叶煊喆、李伟翔

指导教师 毕晓君

2026 年 4 月 6 日

摘要

在光电融合技术飞速发展的后摩尔时代，微环调制器凭借尺寸小、功耗低及易于集成等显著优势，成为下一代光互连系统的核心器件。然而，微环谐振腔对环境热扰动及工艺公差具有极强的敏感性，必须依赖高精度的波长锁定电路维持其稳定运行。传统的纯模拟驱动方案为满足较高的调节精度，面临着芯片面积庞大、动态功耗极高以及在低成本 CMOS 工艺下极易受器件失配限制的严重工程壁垒。

针对上述难题，本项目提出并实现了一款基于 Sigma-Delta 调制技术的高精度波长调节驱动器。该系统架构采用“以数字高频补模拟低位”的设计思路，旨在规避高精度模拟 DAC 的物理设计困境。在系统数字前端，项目设计了插值滤波器 3 阶 4-bit CIFB 结构的 Sigma-Delta 调制器，通过局部谐振零点推移与规范有符号数（CSD）底层硬件优化，将 16-bit 高精度基带信号无损降维为高速 4-bit 码流，并将巨大的量化噪声强制推移至极高频段。在模拟后端，设计了极低开销的 4 位高速电流舵 DAC 结合 OTA 压控恒流源电路，实现了高抗扰度的精确“电压-电流”转换。在最后，本项目基于 Verilog-A 硬件描述语言，构建了包含电热效应与光学谐振特性等物理特性的微环模型。通过该模型验证了微环自身固有的“热惯性”可作为天然的物理低通滤波器，能够自动平滑高频整形噪声，从而在谐振腔内物理重构出极高精度的平均调谐温度。

本项目采取了严格的自顶向下信号设计流程。经 MATLAB 算法综合与 Vivado FPGA 功能验证，数字前端在真实物理时钟与硬件噪声环境下依然稳定实现了 15.98bits 的有效位数，极度逼近纯理论仿真的 16.8bits。在此数字验证基础上，系统将真实高速码流导入 Cadence Virtuoso 环境，驱动后级模拟电路与 Verilog-A 微环模型，完成了跨越数字、模拟与光热物理域的全链路联合仿真。

最终的系统级测试结果表明，该驱动架构不仅成功还原了等效 14.67bits 的平滑加热功率，更在微环光学端实现了 5.3nm 的宽波长调节范围，以及 1pm 波长调节精度。本项目以较小的硬件解决了微环调制器的高精度波长驱动难题，为下一代低成本、高性能光电集成芯片提供了具有应用价值的工程解决方案。

关键词： Sigma-Delta 调制(DSM)；微环调制器；热惯性滤波；波长锁定电路；插值滤波器；

目录

	摘要.....	I
1	背景综述	1
2	研究路线与目标	3
	2.1 研究目标.....	3
	2.2 研究路线.....	3
3	系统架构	5
4	模块电路分析	6
	4.1 插值滤波器设计	6
	4.2 Sigma-Delta 调制器设计	11
	4.3 4 位电流舵型 DAC 设计	16
	4.4 OTA 与功率驱动级设计	20
	4.5 微环调制器 Verilog-A 行为级建模.....	23
5	系统集成仿真与结果分析	25
	5.1 插值滤波器与 DSM 调制器级联 FPGA 功能验证.....	25
	5.2 系统全链路级联仿真与调节指标验证.....	27
6	系统局限性分析与性能优化展望	29
	6.1 系统局限性分析.....	29
	6.2 性能优化展望.....	29
7	参考文献	30
8	附件：设计代码说明	31
	8.1 Matlab 代码	31
	8.2 Verilog HDL 代码.....	33
	8.3 VeilogA 代码	45

1 背景综述

随着半导体工艺不断向物理极限逼近，传统微电子电路在晶体管尺寸缩减上遭遇瓶颈。为了继续推动信息产业的飞速发展，将光学器件与电学器件集成在同一芯片上的“光电融合”技术应运而生，并成为新的发展引擎。相较于传统的金属互连，光电混合集成电路通过光互连技术，实现了信息传递过程中的极低损耗和片上/片间的高速传输，完美契合了当今大数据中心对信号处理“高带宽、低损耗”的严苛需求。

实现光电融合的关键在于可靠的光电调制器。目前广泛采用的传统硅结构调制器（如马赫-曾德尔调制器 MZM），由于其折射率和吸收系数对自由载流子浓度的依赖性较弱，导致器件特征尺寸往往达到数十甚至上百微米，存在占用面积过大、调制功率较高的问题。相比之下，微环调制器展现出了巨大的替代潜力。它具备尺寸小、功耗低、调制效率高以及易于大规模集成等四大核心优势，是当前光电融合领域极具前景的研究方向。

然而，微环调制器存在一个致命的弱点：其谐振腔的谐振波长对环境条件（如温度波动）、制造公差十分敏感；同时，由于入射激光波长本身也存在偏移现象，若不施加有效的人为补偿和控制措施，微环调制器将极难维持在理想的工作状态。

为了克服微环调制器的环境敏感性，设计一款微环波长锁定电路显得尤为重要。该电路的核心作用是完成微环谐振波长的初始校准，并在面对环境热扰动和激光波长偏移时，动态稳定微环的工作状态。

波长锁定电路的核心在于直接作用于微环热驱动器的高精度 DAC，从调节需求的角度出发，满足波长锁定电路的 DAC 精度要求较高。传统纯模拟架构直接物理实现高精度 DAC 的难度较大：一方面是高精度 DAC 所需的面积较大，功耗较高，违背光电集成“小尺寸、低功耗”的初衷；另一方面，在 CMOS 工艺中，器件失配和温度漂移极易破坏高精度 DAC 的线性度，设计难度较高。

为解决上述难题，必须引入数字调制技术，核心思路是“以数字补模拟、以速度换精度”：即使用极低分辨率的物理 DAC，配合高速翻转的数字码流，在时间平均上等效重构出高精度电压。目前主流的调制技术有脉宽调制(PWM)和 Sigma-Delta 调制 (DSM) 两种路线。PWM 技术通过改变固定周期方波的占空比来实现不同电压，其缺点在于，若要实现 N bits 精度，其数字时钟频率必须是输出信号带宽的 2^N 倍，在精度较高时，PWM 的系统时钟会非常高，这样不仅会带来极其严重的动态功耗，且过窄的脉冲在物理开关上极易产生严重的非线性失真。DSM 技术引入高级的噪声整形算法，将带内巨大的截断误差全部推移至高频频段，仅需较低的过采样率即可实现等效的信噪比，而微环调制器的热调节结构本身具有物理上的热惯性，其天然等效为一个截止频率约在 1~10kHz 的物理低通滤波器，能够直接过滤掉被推到高频部分的整形噪声，天然具有互补的性质。因此，团队最后决定采用 DSM 技术。

综上所述,本项目确立了基于 Sigma-Delta 调制技术的高精度驱动架构。在系统数字前端,DSM 模块将高分辨率的目标波长控制信号,动态转化为高速跳变的低位宽数字码流;在模拟物理端,系统仅需配置一个极低成本、小面积的低分辨率高速电流舵型 DAC,即可接收该码流并完成最终的电光转换驱动。

本项目通过严密的理论建模与系统级仿真验证,旨在证明这种“电学数字高阶调制+物理热学自然滤波”的跨界协同架构,能够有效解决传统高精度纯模拟硬件制造的难点。该方案不仅极大降低了前端模拟电路的设计难度,更在有效缩减芯片面积与功耗的前提下,于低成本 CMOS 工艺中实现了完美契合微环调制器要求的高性能波长锁定驱动。

2 研究路线与目标

2.1 研究目标

本项目的核心目标是针对微环调制器波长锁定电路中对高精度、低功耗驱动器的严苛需求，设计并验证一款基于 Sigma-Delta 调制（DSM）技术的高精度混合信号驱动器架构。

具体目标包括：

架构选择:摒弃传统高成本的纯模拟高精度 DAC 方案，构建“数字高阶 DSM 调制+物理 4-bit 电流舵 DAC 输出+微环热惯性低通滤波”的跨界协同系统。

核心算法与硬件设计:在数字前端部分，完成过采样率为 128 倍的多级插值滤波器设计，以及 3 阶 4-bit 的 CIFB 结构 Sigma-Delta 调制器设计；在模拟部分，完成 4bit 电流舵设计以及微环的 Verilog-A 行为级建模。

系统级仿真验证:搭建完整的模数混合系统模型，首先进行各个模块的独立仿真进行功能性验证，其次将插值滤波器与 DSM 级联进行 Verilog HDL 仿真验证数字前端系统设计，最终将数字前端部分与 DAC 及微环建模级联，在 Cadence Virtuoso 上进行数模混合仿真，验证系统在较低过采样率下，能否将量化噪声有效推移至高频，从而在低频基带内实现等效高精度 DAC 的极高信噪比。

2.2 研究路线

为实现上述数模混合的复杂设计目标，本项目采取“算法先行、模块独立验证、全系统级联统调”的设计路线，具体如下：

一、系统级算法建模与参数综合

在架构设计时，确定插值滤波器与 DSM 的架构规格。利用 MATLAB SD Toolbox 综合 3 阶 4-bit CIFB 调制器的噪声传递函数，优化局部谐振零点，完成动态范围缩放，提取出可用于硬件实现的 CSD 编码系数，并在 Simulink 中搭建 DSM 行为级模型，通过频谱分析（PSD）初步验证算法层面的信噪比与噪声整形效果；在 Vivado 中完成插值滤波器的设计，并进行独立的功能仿真验证。

二、数字前端 RTL 设计与 FPGA 验证

用 Verilog 语言编写 DSM 调制器的底层硬件代码，然后将滤波器与 DSM 级联，生成比特流并下发至真实 FPGA 开发板，利用集成逻辑分析仪（ILA）抓去数字码流信号后将数据导入 Matlab 进行分析，确保数字逻辑在实际硬件时序下的正确性。

三、模拟电路设计与微环物理建模

在 Cadence Virtuoso 中，采用 65nm 工艺，设计 4bit 高速电流舵型 DAC 晶体管级电路以及之后的 OTA，并根据微环热调节器的物理参数，通过 Verilog-A 实现微环的物理建模。

四、全链路混合信号联合仿真

将数字前端设计导入 Cadence Virtuoso 仿真环境中，与 DAC 及微环建模进行全系统级联，通过混合信号瞬态仿真观测微环模型末端最终输出的平均加热功率/等效电压波形，以最终的系统级结果验证该“高阶数字调制+低位模拟输出+物理热学滤波”架构的完备性与高精度性能。

3 系统架构

为实现高精度的微环波长调节，本项目设计了一套数字与模拟深度融合的驱动系统。该系统架构如图 3.1 所示，信号的数据流向与各核心模块协同机制如下：

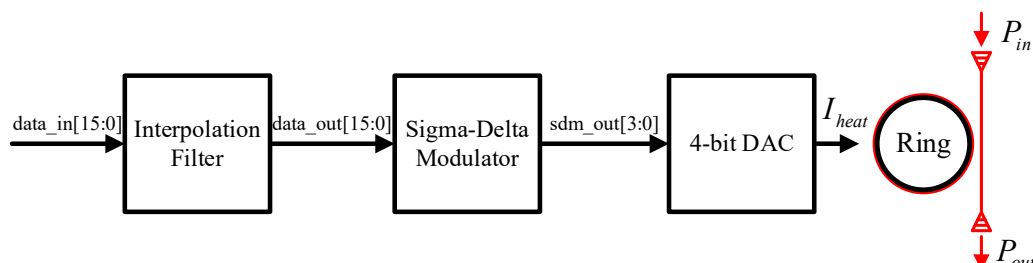


图 3.1 系统架构图

一、数字前端：插值滤波器

作为系统的数据入口，插值滤波器负责接收来自微控制器的高精度（16-bit）、低采样率波长控制信号。其核心功能是过采样，将信号采样率提升数十倍至系统高速时钟频率；同时，它相当于一个数字低通滤波器，负责滤除上采样过程中产生的频谱镜像，为后续调制器提供高速、平滑的高质量数据流。

二、数字核心：Sigma-Delta 调制器（DSM）

本系统实现“降维与整形”的运算核心。接收到高速高精度信号后，DSM 通过其内部的 3 阶级联积分器和反馈网络，将 16-bit 信号强制截断为高速跳变的 4-bit 数字码流。在此过程中，DSM 强大的噪声整形机制将产生的巨大截断量化噪声全部“推扫”至极高频频段，从而在低频目标带宽内保留了极度纯净的信号能量。

三、模拟设计：4 位高速数模转换器与 OTA

DAC 负责接收来自数字前端的高速码流，并将其转换为对应的阶梯状模拟控制电压，从宏观上使低频均值对应目标设定值；之后通过 OTA 将电压信号转换为驱动电流信号流入下一级微环，从而实现对接波长的调控。

四、物理建模：Verilog-A 微环建模

利用物理公式，在 Cadence Virtuoso 中实现 Verilog-A 微环建模，OTA 驱动后的电流加热上方电阻产生焦耳热，进而通过热光效应改变微环的折射率完成对输出光功率波长的锁定调谐。

4 模块电路分析

4.1 插值滤波器设计

4.1.1 插值滤波器整体设计

在高性能光电微环驱动系统中，为了实现皮米（pm）级的波长调节精度，后端 Sigma-Delta 调制器与电流舵 DAC 要求极高的过采样率（OSR）。本项目设定系统总过采样率为 128，旨在将信号基带内的量化噪声推向高频，并通过数字滤波手段彻底滤除镜像频率分量。

为了实现 128 倍的升采样率，本设计摒弃了硬件开销巨大的单一结构滤波器方案，构建了由 2 倍半带滤波器、4 倍 1/4 带滤波器及 16 倍保持采样组成的多级级联链路。为了进一步优化底层硬件实现，设计中摒弃了传统的硬核乘法器，转而对滤波器系数进行了 CSD 编码优化，采用移位器与加法器组合，在确保信号处理性能的同时显著提升了资源的利用率。

在链路前端，第一级与第二级滤波器工作在较低的采样频率下，这使得系统可以采用较精细的 FIR 结构来保证通带平坦度与过滤精度。这一阶段的处理能够以较低的硬件代价，在信号进入高速处理域之前彻底滤除基带附近的镜像干扰，为后续的高精度信号重构奠定了坚实基础。

随着采样率通过前两级提升了 8 倍，镜像频率已被推向高频远端，此时滤波器对过渡带陡峭度的要求大幅降低。因此，在采样率最高的末级，设计采用了完全不含算术运算的 16 倍保持采样结构。通过这种由精入简、逐级加速的级联设计，系统仅需少量资源便实现了优异的带外抑制效果。具体的数字信号处理（DSP）逻辑流向及级联架构如图 4.1.1 所示：

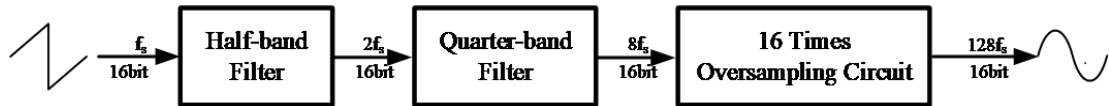


图 4.1.1 插值滤波器整体设计

4.1.2 半带滤波器设计与实现

作为级联插值链路的首级，第一级半带滤波器（HB）旨在 50kHz 采样频率下实现高精度的信号提取与镜像抑制。本设计采用等波纹 FIR 结构，充分利用半带滤波器系数的对称性，通过移位加法逻辑实现。

在指标规划上，本级滤波器承担了最严苛的噪声抑制任务。为确保系统量化精度不受镜像分量折叠的影响，设计设定阻带抑制目标为 90dB，实测值达到 93.57dB，使残留噪声远低于系统的量化噪声水平。同时，为保障微环调制器反馈控制的线性度，通带纹波被严格控制在 0.00025dB 以内（对应幅度小于 1LSB），具体的物理指标与仿真结果见表 4.1.1:

表 4.1.1 第一级半带滤波器性能指标汇总

参数	仿真结果
输入采样率	25kHz
工作采样率	50kHz
滤波器阶数	101
通带截止频率	11.012kHz
阻带起始频率	13.988kHz
阻带最小抑制	93.57dB
通带纹波	0.000229dB

为进一步验证设计指标的可靠性，通过 MATLAB 对该 101 阶半带滤波器的幅频响应进行了仿真分析。如图 4.1.2 所示，系统在 0.00025dB 的严苛约束下，依然保持了极佳的通带平坦度，有效避免了调节信号的幅度失真。同时，曲线在过渡带表现出极高的陡峭度，阻带内的抑制深度稳定保持在 90dB 以下。

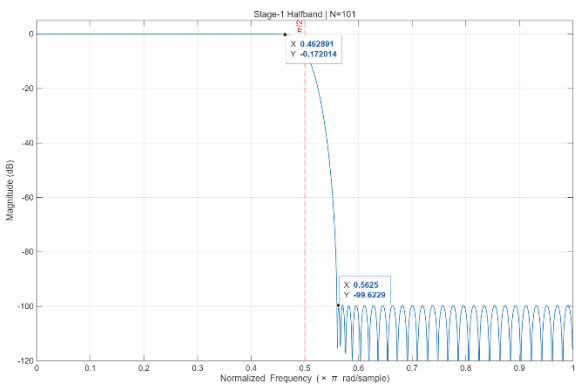


图 4.1.2 半带滤波器幅频特性

4.1.3 1/4 带滤波器的设计与实现

在首级半带滤波器完成 2 倍升采样后，级联链路进入第二级 1/4 带滤波器处理阶段。该模块工作在 200kHz 的采样频率下，核心任务是进一步抑制由 4 倍内插引入的镜像分量，并为末级的高倍率保持采样提供纯净的带内信号。本设计采用了 41 阶等波纹 FIR 结构，通过对滤波器阶数的精简，有效缓解了采样率提升对硬件时序收敛带来的压力。具体物理指标与测试结果见表 4.1.2：

表 4.1.2 第二级 1/4 带滤波器性能指标汇总

参数	仿真结果
输入采样率	50kHz
工作采样率	200kHz
滤波器阶数	41
通带截止频率	11.000kHz
阻带起始频率	38.000kHz
阻带最小抑制	97.42dB
通带纹波	0.000231dB

针对该 41 阶滤波器的幅频特性仿真如图 4.1.3 所示。曲线表明，系统在 200kHz 采样率下保持了优异的频率选择性。

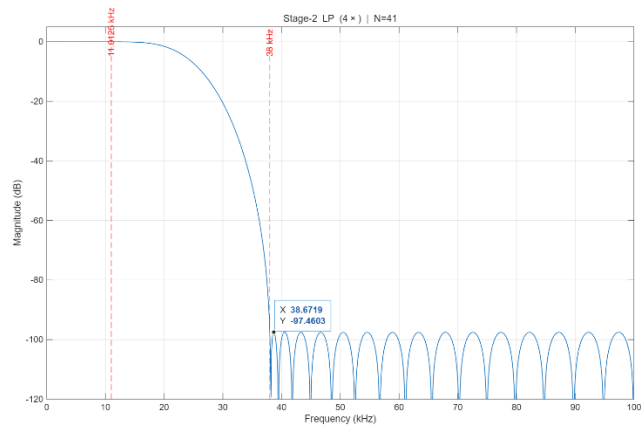


图 4.1.3 1/4 带滤波器幅频特性

4.1.4 滤波器级联性能评估

在完成半带滤波器与 1/4 带滤波器的级联设计后，本设计对其综合性能进行了仿真验证。如图 4.1.4 所示，在实现高效镜像抑制的同时，确保了基带信号的高保真传输。

仿真结果表明，在 16-bit 量化位宽约束下，系统带内信噪比达到 103.80dB。级联后的累计通带纹波仅为 0.0004dB，极大压制了幅度失真，确保 0-10kHz 基带信号的转换精度。

在频谱特性方面，级联链路在 40kHz 以上阻带范围内表现出较强的抑制能力，最差抑制点稳定保持在 97.4dB。这证明在仅通过前两级滤波处理的情况下，系统便已能彻底滤除内插产生的镜像分量，为整体链路提供了极低噪声的数字基带环境。

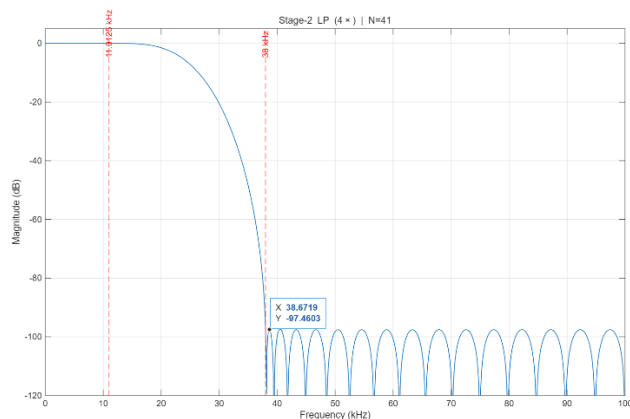


图 4.1.4 滤波器级联幅频特性

4.1.5 Simulink 系统级仿真

为了验证上述滤波器设计在全链路中的实际工作表现，本设计在 Simulink 环境下构建了系统级仿真模型（如图 4.1.5 所示）。仿真链路完整模拟了信号从基带输入到 128 倍插值输出的全过程：首先通过“零值内插”提升采样率，随后利用设计好的半带滤波器与 1/4 带滤波器逐级滤除镜像干扰，最后通过 16 倍保持采样模块完成最终的速率匹配。

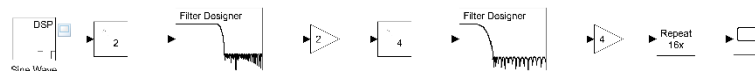


图 4.1.5 128 倍内插链路系统级仿真建模图

全链路各阶段的时域波形如图 4.1.6 所示。从波形演变过程可以直观观察到插值滤波算法对信号质量的显著提升。初始输入信号由于采样频率较低，呈现出明显的阶梯状失真；经过首级 2 倍插值与半带滤波后，波形阶梯变得密集，初步显现正弦轮廓。随后，信号进入 4 倍插值与 1/4 带滤波阶段，呈现出连续平滑的正弦形态。末级 16 倍保持采样进一步加密了采样点，使最终输出波形具备极高的平滑度与时间分辨率。

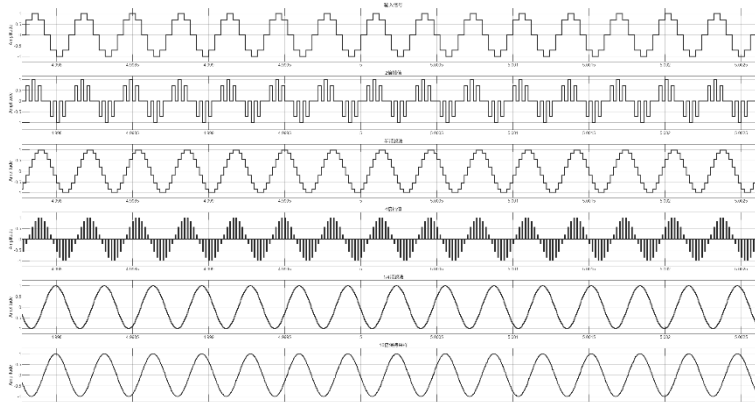


图 4.1.6 系统各级输出信号的时域波形演变图

此次系统级仿真不仅验证了各级滤波器算法的时域有效性，更证明了级联插值架构在处理高倍率内插任务时的稳定性。时域波形的平滑演进与前述频域分析结果高度一致。

4.1.6 Verilog 实现与仿真

在完成系统级建模后，本设计采用 Verilog HDL 对多级插值滤波器进行了 RTL 级硬件描述，并通过 Modelsim 进行了时序仿真。为了验证硬件电路处理信号的精确度，将仿真输出的数字序列导入 MATLAB 进行处理分析。

图 4.1.7 展示了底层电路在不同阶段输出的时域信号波形。实验结果表明，硬件实现的波形演变与 Simulink 行为模型高度一致。

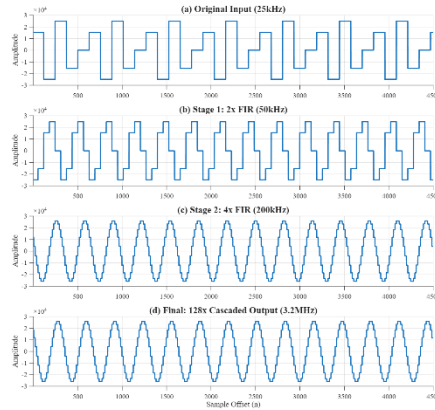


图 4.1.7 基于 Modelsim 的硬件各级采样波形仿真图

为进一步评估硬件级设计的频谱纯度，对 Modelsim 导出的 128 倍插值数据进行了功率谱密度（PSD）分析，结果如图 4.1.8 所示。在输出频率为 10kHz 的测试条件下，信号的主频峰值清晰，带内噪声基底抑制在-120dB 以下。图中标记显示，带外主要分量的抑制深度达到了-105.335dB，这验证了硬件电路实现较好的阻带衰减特性。

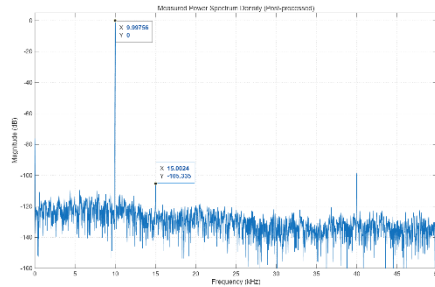


图 4.1.8 Modelsim 硬件仿真输出信号频谱特性图

该分析从 RTL 级层面确认了插值滤波器在滤除高频镜像和维持信噪比（SNR）方面的性能，为后续通过三阶 DSM 调制器产生高质量码流提供了高保真的硬件前端支撑。

4.2 Sigma-Delta 调制器设计

前级的插值滤波器输出的是高精度 16bit 高精度波长调节信号。在实际的半导体工艺中，受限于匹配精度，在模拟领域直接实现 16-bit DAC 不仅占用面积巨大，且极易受到温度和工艺偏差的影响，难以实现。为了处理此类问题，本项目在插值滤波器和后级 DAC 中插入过采样 Sigma-Delta 调制器（DSM），利用过采样（Oversampling）和噪声整形（Noise Shaping）技术，将高分辨、高采样率的输入信号，重新量化为低分辨率（4bits）的数字码流，同时利用 DSM 内部的积分反馈环路，将截断产生的量化噪声推向远离基带信号的高频频段，从而有效保护低频波长控制信号的信噪比。

由前面的插值滤波器设计可知，输入信号的信号带宽为 10KHz，信号时钟频率为 3.2MHz，过采样率为 128。同时，对于 16 位调制器结构，根据公式 $SNR = 6.02N + 1.76$ 可以推出，SNR 的值为 98.08dB，考虑到模拟重构需要留有一定的设计裕度，设计时应尽量要求要求信噪比大于 105dB。

表 4.2.1 sigma-delta 调制器的设计指标

性能	指标
输入信号时钟 (f_s)	3.2MHz
输入信号带宽 (f_b)	10kHz
过采样率 (OSR)	128
信噪比 (SNR)	>105dB

根据设计指标，需要确定调制器的结构、阶数和量化器位宽：

1.量化位宽

调制器常见的量化方式为单比特和多比特。对于单比特量化输出而言，该设计结构较为简单，且线性度较高，但输入和输出的增益不确定，容易导致积分器饱和、环路不稳定，且底噪过大，难以满足设计要求。而多比特量化能够有效提升信噪比，每增加 1 bit 量化位宽，系统的信噪比可提升约 6dB，且能显著增强高阶环路的稳定性。在综合考虑设计需求和难度后，最终选择 4bit 量化输出。

2.调制器结构

DSM 调制器有三种典型结构，分别是 MASH（多环级联）、CIFI（前馈）以及 CIFI（分布式反馈）。其中 MASH 结构的成功较大依赖于各级之间的匹配，在实际的制造工艺中容易因为工艺偏差而性能大幅下降，影响信噪比；CIFI 结构有信号摆幅小，降低功耗的好处，但是该结构的传递函数通常在带外有较为明显的尖峰增益，容易放大前一级插值滤波器的高频镜像噪声，影响输出；而 CIFI 结构既有天然的低通滤波特性，能够有效压制高频噪声，同时采用分布式架构能够满足较高的时钟频率（过采样率）以及较少的资源消耗，契合本项目的需求。

最终采用 CIBF 结构。

3.调制器阶数

调制器的阶数直接决定了噪声整形滤波器对量化噪声的高频推移能力。在 $OSR=128$ 的条件下，1 阶或 2 阶调制器的噪声整形斜率较缓，难以在 10kHz 带宽内实现 $>105\text{dB}$ 的信噪比；而 4 阶及以上的系统虽然理论精度极高，但存在严重的条件稳定性问题，极易发生极限环振荡。因此，3 阶系统是实现本项目较为理想的阶数设置。

在上述分析后，该项目最终选择 3 阶 4bit 量化的 CIBF 结构 DSM 调制器，在 Matlab simulink 里建模如下：

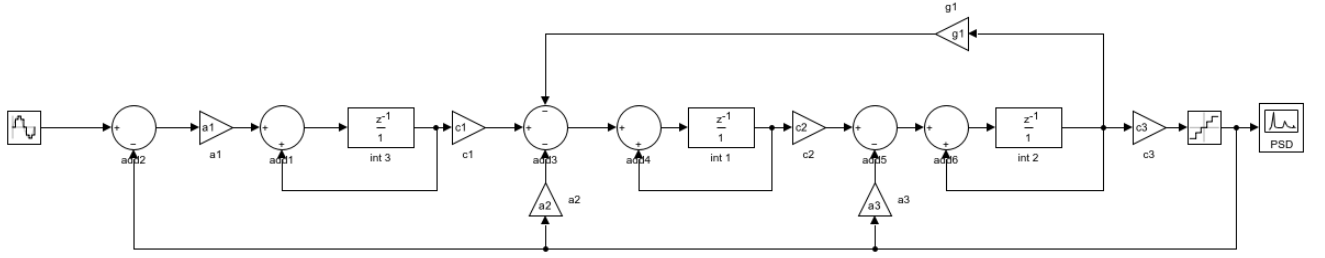


图 4.2.1 3 阶 4bit 量化 CIBF DSM 结构图

该结构具有三个离散时间积分器，在时域上对信号进行累加，最终结果在系统末端的多位量化器作用下截断为离散电平，产生的量化误差通过全局误差反馈环路反馈至每一级积分器的输入端进行积分，不断减小误差。同时，在结构中还引入了局部负反馈系数 g_1 ，将一个传输零点推移至 10kHz 带宽内的特定高频处，从而进一步提高带内信噪比。

利用 Matlab 中的 Delta-Sigma Toolbox 工具来设计噪声传递函数，具体实现的 Matlab 代码见附录。运行代码后，得到该调制器的零极点为

$$\begin{aligned} z_1 &= 1.0000 & z_{2,3} &= 0.9983 \pm 0.0581i \\ p_1 &= 0.6289 & p_{2,3} &= 0.7242 \pm 0.3106i \end{aligned}$$

因此噪声传递函数 $NTF(z)$ 为

$$NTF(z) = \frac{(z-1)(z^2 - 1.9966z + 1)}{(z-0.6289)(z^2 - 1.4484z + 0.6209)}$$

传递函数 $STF = 1$ ，即输入信号在全频带内无衰减。仿真得到噪声传递函数的零极点如图 4.2.2。其中 $\times$ 代表极点， $\circ$ 代表零点。从图中看出三个极点均严格位于单位圆内部，保证了系统的绝对稳定性；一个零点位于 $z = 1$ ，另外两个零点作为一对共轭复根，均匀地分布在 $z = 1$ 附近的单位圆上，有效压平信号带内的量化噪声，提升高阶系统的稳定性。

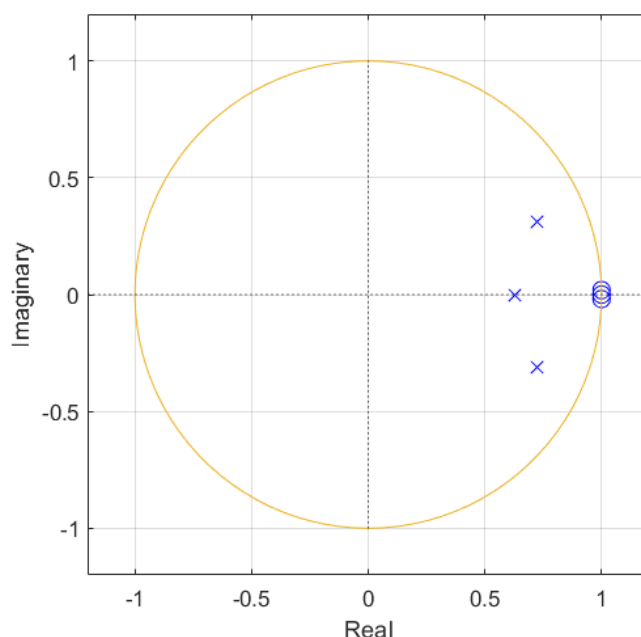


图 4.2.2 噪声传递函数零极点图

上述代码运行后，得到各个系数如下图所示。由于考虑到最终实际硬件实现的可行性,最终的参数将通过 CSD 编码减少系数非零位个数，减少硬件消耗，量化前后的值如下：

系数名称	量化前	CSD 编码实现	量化后	误差
$a_1 = b_1$	0.010079	$2^{-7} + 2^{-9} + 2^{-11}$	0.010254	1.73%
a_2	0.023481	$2^{-5} - 2^{-7} + 2^{-11}$	0.023926	1.89%
a_3	0.041880	$2^{-5} + 2^{-7} + 2^{-9}$	0.041016	2.06%
c_1	0.395774	$2^{-1} - 2^{-3} + 2^{-6} + 2^{-7} - 2^{-10}$	0.397461	0.43%
c_2	0.728507	$2^{-1} + 2^{-2} - 2^{-5}$	0.718750	1.34%
c_3	22.03289	$2^4 + 2^2 + 2^1$	22.000000	0.15%
g_1	0.000496	$2^{-11} + 2^{-17}$	0.0004959	0.02%
$b_2 = b_3$	0	0	0	0

表 4.2.2 DSM 调制器各系数量化前后值对比

确定参数后，在 Simulink 中运行仿真，给调制器施加幅度为 0.5，频率为 10KHz，采样频率为 3.2MHz 的正弦波信号，得到波形和输出信号频谱图 4.2.3 和图 4.2.4。从图 4.2.3 看出，该结构实现了 4bit 量化的作用，成功输出码流。对调制器输出做 FFT 分析，得到图 4.2.4 的结果，可以看出，在信号带宽内，信噪比为 123.9dB，有效位数为 20.28 位，满足设计要求，证明该方案可行性。

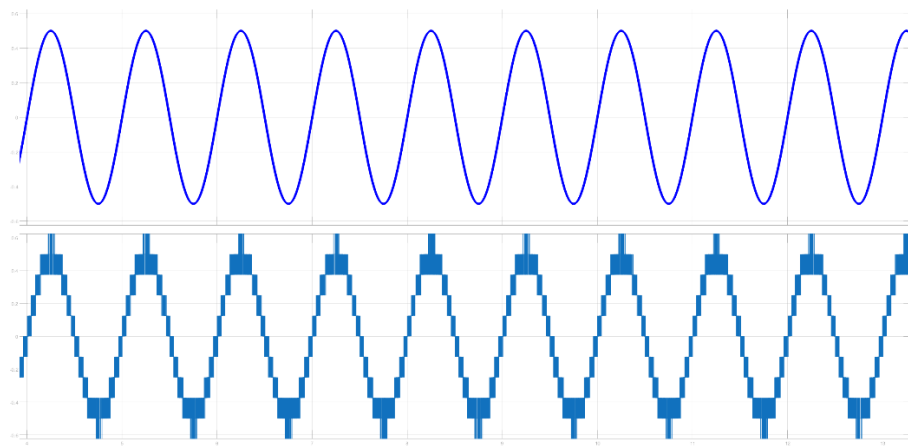


图 4.2.3 DSM Simulink 时域仿真

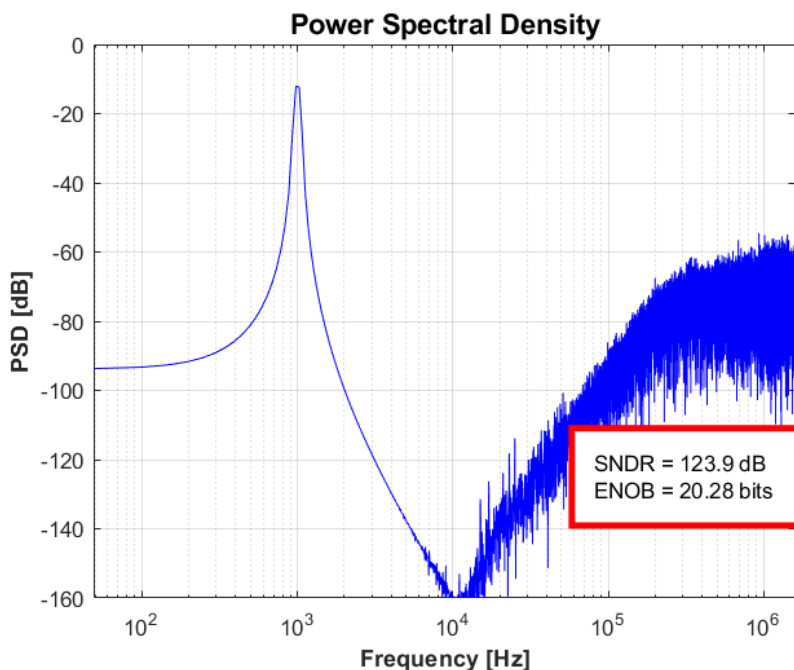


图 4.2.4 调制器输出信号频谱图

在 Matlab 仿真完成后，再将该调制器利用 Verilog HDL 实现仿真，输入信号为 10kHz、3.2MHz 采样率、幅值为 0.8 的理想正弦波，在 Vivado 平台上进行行为级逻辑仿真后，得到输出数据后导入 Matlab 进行分析，得到相应的频谱和时域图像。

在处理数据时，由于 DSM 调制器在前段部分没有达到稳定状态，在共产生的 1043989 个样本点里面，我们舍弃前 10 万个点，并选用 $\beta = 14$ 的 Kaiser 窗进行频谱仿真，得到如下图像。其中频谱图表明在行为级仿真中该调制器信噪比达 107dB，有效位数为 17.04 位，再次验证设计的可行性。

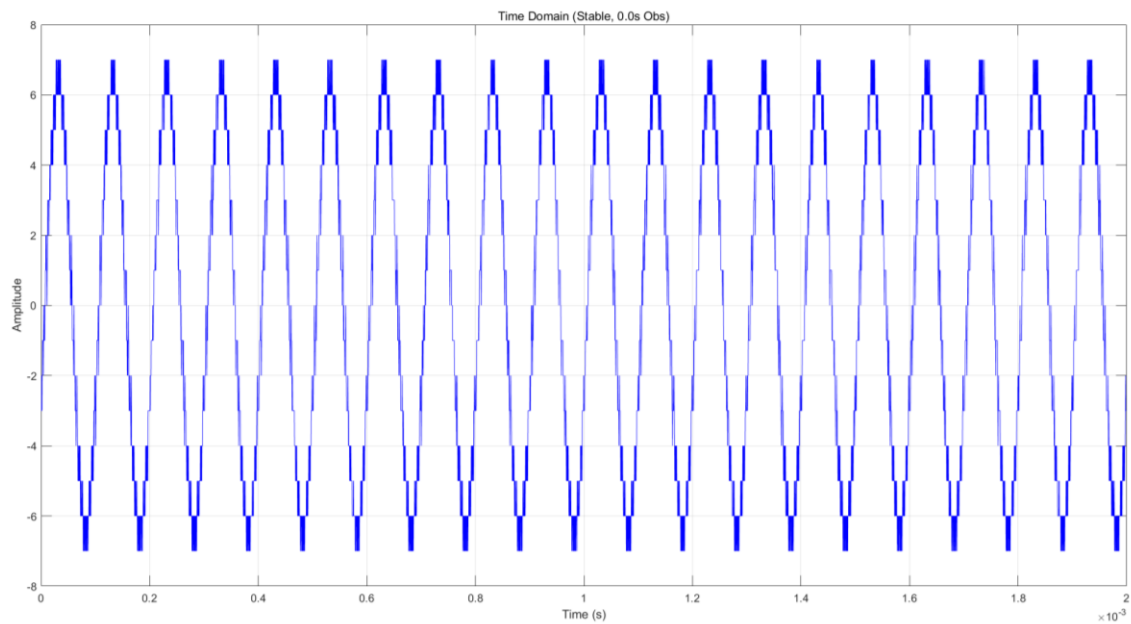


图 4.2.5 DSM 单级 FPGA 仿真时域图

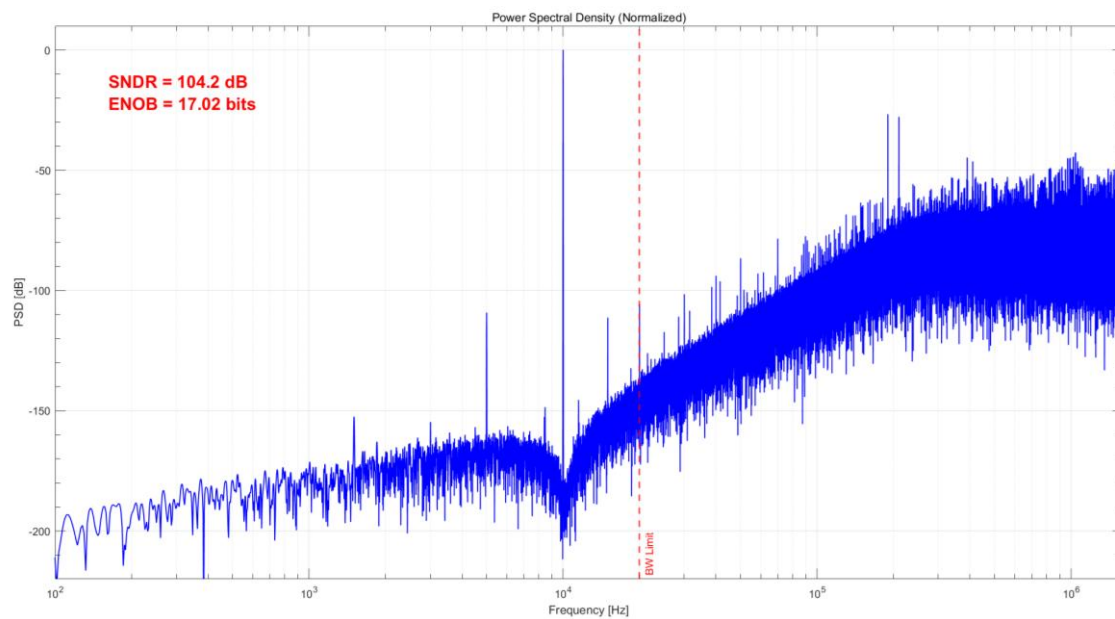


图 4.2.6 DSM 单级 FPGA 仿真频谱图

4.3 4 位电流舵型 DAC 设计

前端 Sigma-Delta 调制器的输出是一组包含大量高频整形噪声的高速 4bit 数字码流。为了处理这种具有极高数据翻转率的信号,传统的电阻串型或者电容阵列型 DAC 存在建立时间长、寄生电容大等问题。为了满足高速瞬态响应的要求,本项目选择电流舵型架构,利用高速电流开关直接控制电流流向,具有较高的转换速度。

在数字编码方式上,对于传统的二进制编码中,由于各位权重不同,在发生如 0111 到 1000 的进位跳变时,所有开关都会发生反转,极易在输出端产生由与翻转时间差异造成的动态毛刺,从而破坏性能。本项目采用了全温度计码 (Thermometer Code) 结构,将 4bit 二进制码完全译码为 15bit 温度计码。温度计码的特征在于其各位权重完全相等,当数字量每增加 1LSB 时,仅有一个开关闭合,有效地将开关切换时的动态毛刺降低,显著优化了积分非线性 (INL) 和微分非线性 (DNL) 的指标。

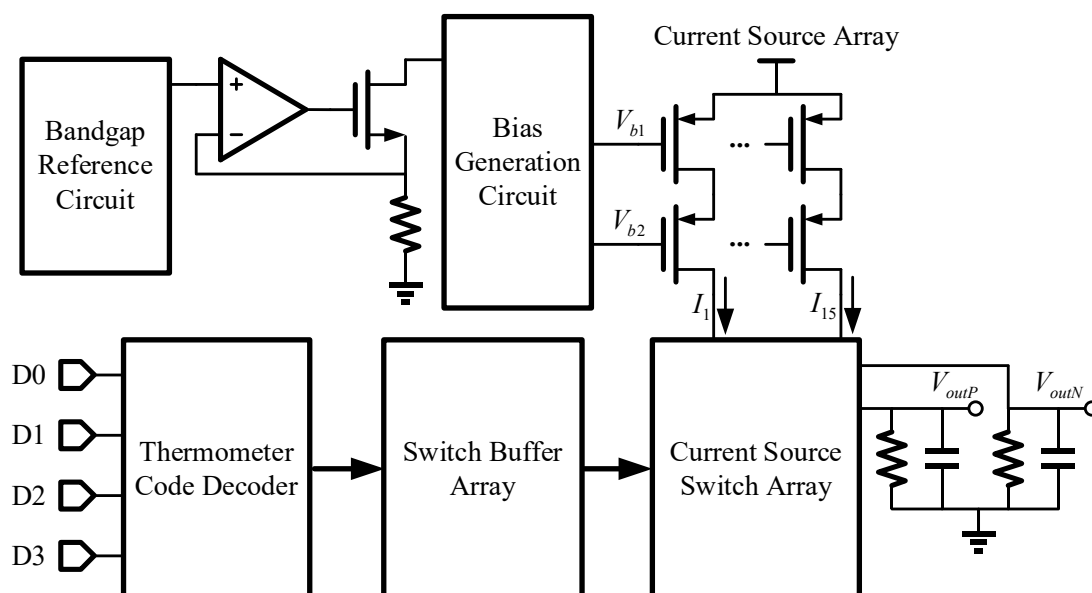


图 4.3.1 7 4bitDAC 系统架构

如图 4.3.1 7 所示, 4bit 温度计码电流舵型 DAC 主要由三个模块组成:

1.译码与时序缓冲电路

采用组合逻辑将输入的 4bit 二进制码转换为 15 位温度计控制信号。随后, 控制信号进入交叉同步缓冲器。缓冲网络被专门设计为具有“先断后通”的时序特性, 确保差分开关在状态切换时, 不会出现两条支路同时导通或同时关断的重叠死区, 从而最大程度降低时钟馈通效应。

2.单位电流阵列

由基础带隙基准 (Bandgap Reference) 提供偏置电压, 并通过 PMOS 电流镜阵列镜像出 15 个完全相同的单位电流源 (Unit Current Source)。为保证精度, 电流源晶体管选用了较大的沟道长度和栅宽, 以减小由于工艺阈值电压失配引起的电流偏差, 并提高输出阻抗。

3.差分开关阵列

每个单位电流源连接一对 PMOS 差分开关。根据温度计码的状态，恒定电流被导向输出节点 V_{outP} 或者伪负载节点 V_{outN} ，确保电流源始终处于导通状态，消除了电流源开关时的延迟。

首先隔离数字前端，单独验证 4-bit DAC 的性能指标。在 Cadence Virtuoso 中构建理想数字激励 Testbench。输入端施加从 0000 到 1111 的理想数字阶梯步进信号。观察 DAC 模拟输出端在 $1k\Omega$ 负载电阻上的瞬态电压响应，如图 4.3.所示。将仿真结果数据导出，通过 MATLAB 脚本处理数据，从而计算得到最小有效位的电压步长、满量程输出范围、积分非线性性和微分非线性误差等性能指标。

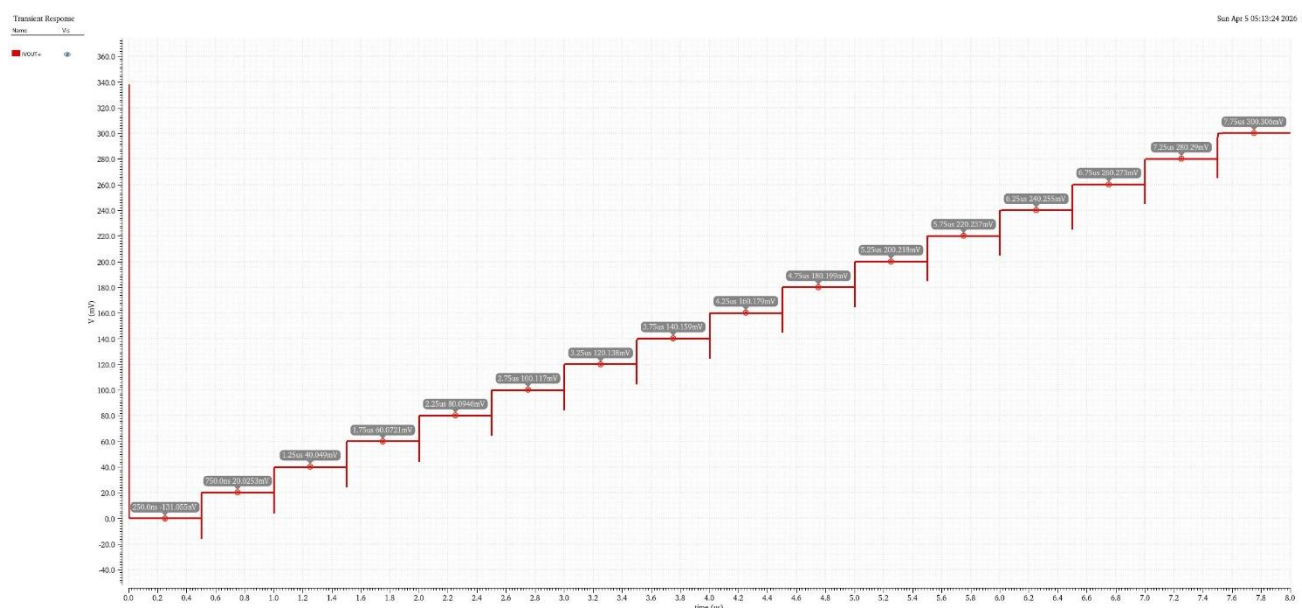


图 4.3.2 DAC 在理想数字阶梯步进信号下的瞬态响应

计算得到 DAC 的性能指标如表所示：

性能指标	符号	仿真结果	单位
满量程输出范围	V_{FS}	0~300	mV
最小有效步长	V_{LSB}	20.0202	mV
微分非线性误差	DNL	0.0002	LSB
积分非线性误差	INL	0.0008	LSB
建立时间	$T_{setting}$	0.2	ns

表 4.3.1 DAC 性能指标

由表 4.3.1 所示，DAC 的两种非线性误差均小于 0.001LSB，表现出极高的线性度，为下一步数模混合仿真奠定了基础。接着将 DAC 与前端使用 Verilog 编写的 Sigma-Delta 调制器以及插值滤波器进行级联，旨在验证在真实高频码流下系统的动态解析能力。为了进行动态性能评估，激励模块的系统主时钟频率为 3.2MHz，内部每过 128 个时钟周期更新一次数据，

等效基带采样率为 25kHz。模块生成了一个频率为 10kHz、幅度为 26214 的 16bit 的高精度正弦数字信号，送入 DSM 中进行调制后转化为 4bit 脉冲密度调制码流（PDM）并直接驱动 DAC，在模拟域中还原出正弦波形，如图 4.3.所示。

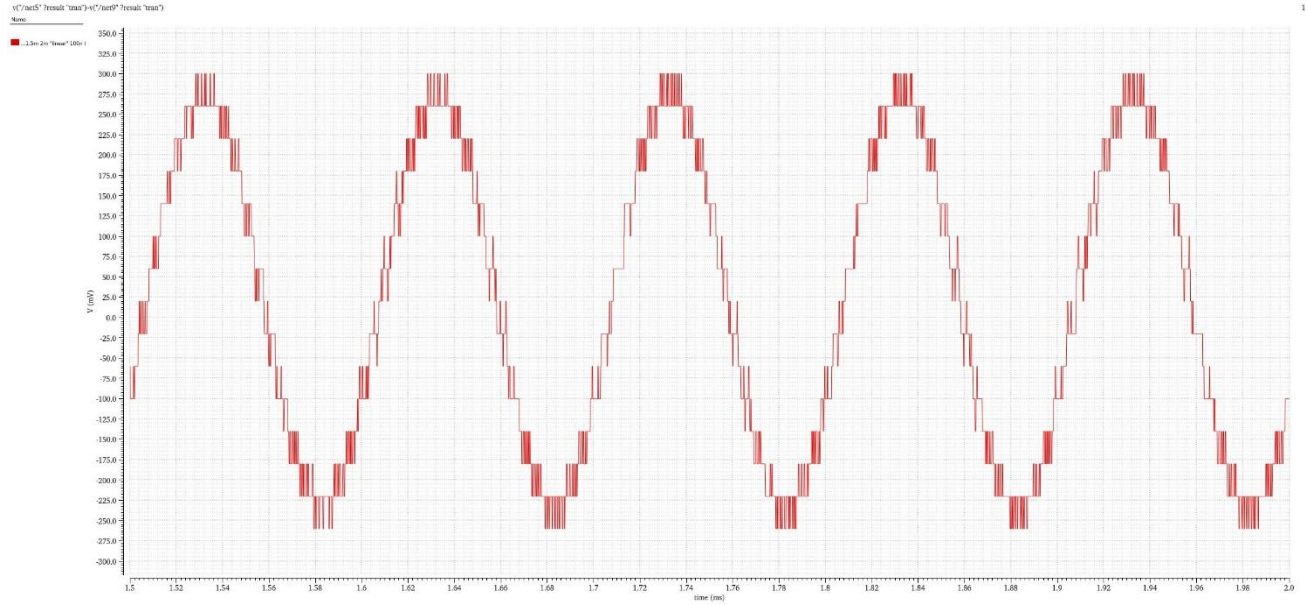


图 4.3.3 DAC 输出调制波形

从 Cadence Virtuoso 导出 DAC 的瞬态电压波形至 MATLAB 中进行离线分析。为了获得精确的动态响应指标，在后处理算法上进行了物理与数学修正：

1.消除瞬态建立误差

在提取瞬态数据中，强制丢弃前 30000 个采样点。此时系统已经度过了上电与反馈环路的建立时间，确保进入绝对的稳态输出阶段，排除了瞬态非理想效应对底噪的干扰。

2.抑制频谱泄露

为了防止非相关采样带来的频谱能量泄露掩盖真实的量化噪声，对截断后的时域数据数据加注了 Kaiser 窗函数。特别地，选取了较大的形状参数 ($\beta = 14$)，以极高的旁瓣衰减换取极低的频谱本底干扰。同时，在频域幅度计算中引入相干增益补偿，以还原真实的信号功率。

在功率谱密度分析中，对于信号功率(P_s)的计算，本项目的有效分析基带带宽设置为 $BW = 20kHz$ 。由于 Kaiser 窗会导致主瓣展宽，单独读取峰值会产生误差。算法以 10kHz 为中心，向两侧延展了 20 个频率点进行主瓣能量积分，囊括了所有的信号能量。对于带内噪声功率 (P_n) 的计算，在 20kHz 基带带宽内进行积分，并严格剔除了直流分量以及上述被标记为主瓣信号的频点，提取出纯粹的带内量化噪声和热噪声功率。最终根据积分得到信号与噪声功率，计算系统信噪比 (SNR) 与等效有效位数 (ENOB)：

$$SNR = 10 \log_{10} \left(\frac{P_s}{P_n} \right)$$

$$ENOB = \frac{SNR - 1.76}{6.02}$$

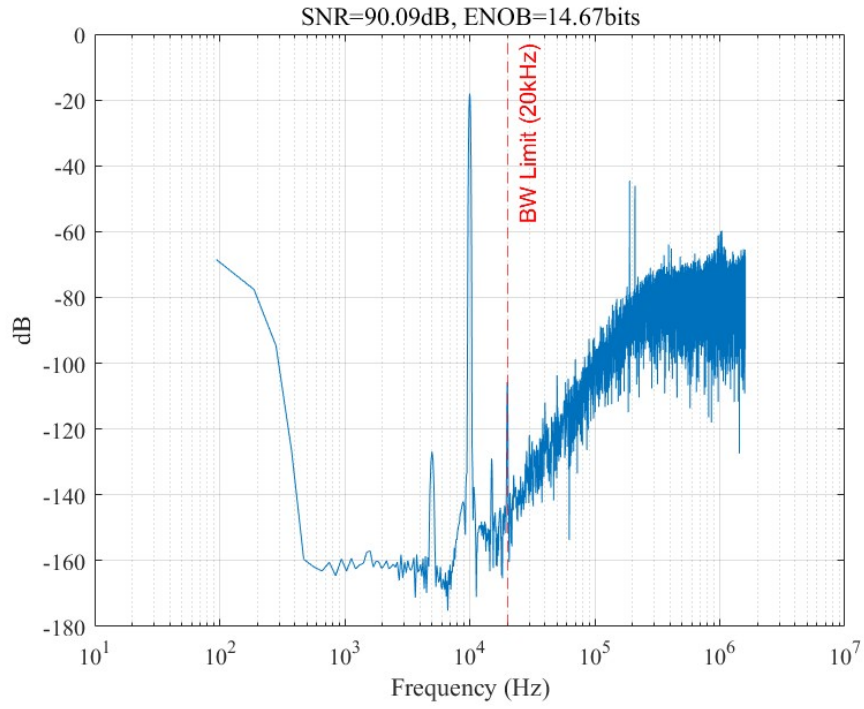


图 4.3.4 频谱分析图

根据频谱分析图（图 4.3.），最终分析得到信噪比为 90.09dB，ENOB 为 14.67bits。尽管物理上采用的是 4bit 粗分辨率 DAC，但系统在闭环基带内成功实现了接近 15bit 的超高精度模拟重构能力，为后面微环的精细波长调节奠定基础。

4.4 OTA 与功率驱动级设计

前级 DAC 已经能够完成数字到模拟信号的转换，但其单端输出电压范围有限，难以直接满足微环热调所需的驱动能力。在光电微环系统中，驱动微环产生较大波长漂移 ($>5\text{nm}$) 需要耗散数十毫瓦的热功率。微环加热器的等效阻抗较低 (约 70Ω)，故需要高达数十毫安的驱动电流。前级 DAC 产生的 $0\sim 300\text{mV}$ 的电压信号无法直接驱动微环，因此需要设计功率放大电路。功率放大级和 DAC 的结构如图 4.4.所示。

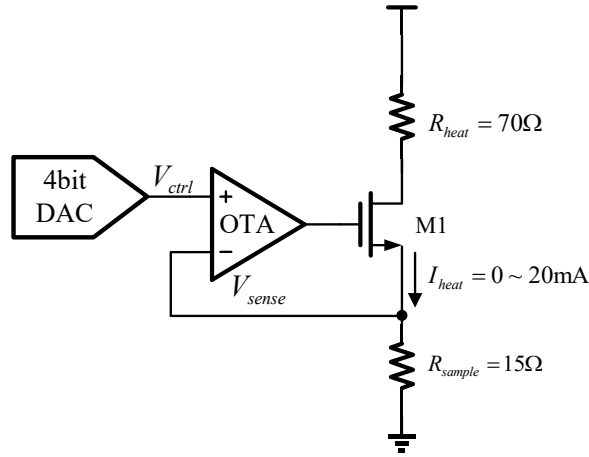


图 4.4.1 功率驱动电路原理图

为实现功率放大本项目应用了一种闭环驱动架构。OTA 的正相输入端接收 DAC 输出的高精度参考电压 V_{ctrl} ，功率 NMOS 导通后，流经微环加热器的电流 I_{heat} 必然流经采样电阻 R_{sample} ，在 OTA 的反相输入端产生反馈电压。由于 OTA 具有极高的开环增益，反馈电压跟随正相输入 $V_{sense} \approx V_{ctrl}$ ，最终加在微环加热电阻上的功率为：

$$P_{heat} = \left(\frac{V_{ctrl}}{R_{sample}} \right)^2 \frac{1}{R_{heat}}$$

基于上述闭环拓扑，OTA 设计必须满足，输入共模范围需要贴近零点位，并且需要稳定驱动功率 NMOS 的栅极电容。为此，本项目设计了一款单级折叠 Cascode 放大器，通过 PMOS 差分输入保证共模输入范围可以满足低电位要求，折叠 Cascode 可以增加开环增益。放大器电路原理图如图 4.4.4.4.2 所示。

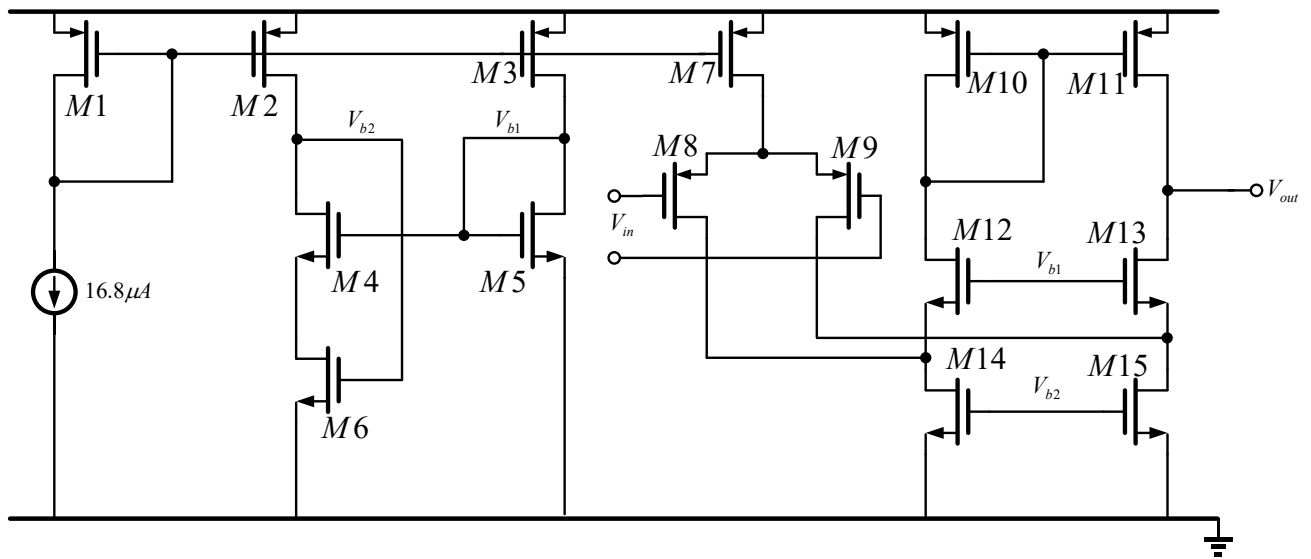


图 4.4.2 折叠 Cascode 放大器原理图

放大器的输入共模信号下限表达式如下，可以达到低电位的要求：

$$V_{in,min} = V_{SS} - |V_{THP}| - |V_{Dsat,tail}|$$

放大器的增益表达式近似为：

$$A_v = g_{m,in} \times (g_{m,cas} r_{o,cas} r_{o,tail}) || r_{o,mirror}$$

对放大器进行 AC 仿真，幅频响应和相频响应如图所示，根据仿真结果，放大器的性能指标整理为

表 4.4.1。由表可知，放大器的性能可以满足功率放大电路的需要。

性能指标	符号	仿真结果	单位
开环增益	A_{vOL}	44.56	dB
相位裕度	PM	41.45	(°)
增益带宽积	GBW	56.08	MHz
输入共模范围	ICMR	0~1.48	V

本节设计了用于微环热调节驱动的 OTA 及其闭环功率放大结构。该电路采用差分输入、单端输出 OTA 作为核心放大单元，通过偏置网络建立稳定工作点，在负反馈电压跟随配置下实现对 DAC 输出控制电压的精确复制，并进一步驱动后级功率支路输出更大的加热电流。相比仅使用 DAC 直接驱动负载，该方案在保持控制精度的同时显著提升了驱动能力，为实现微环波长的大范围、高精度调节提供了必要的模拟接口。

性能指标	符号	仿真结果	单位
开环增益	A_{vOL}	44.56	dB
相位裕度	PM	41.45	(°)

增益带宽积	GBW	56.08	MHz
输入共模范围	ICMR	0~1.48	V

表 4.4.1 OTA 性能指标

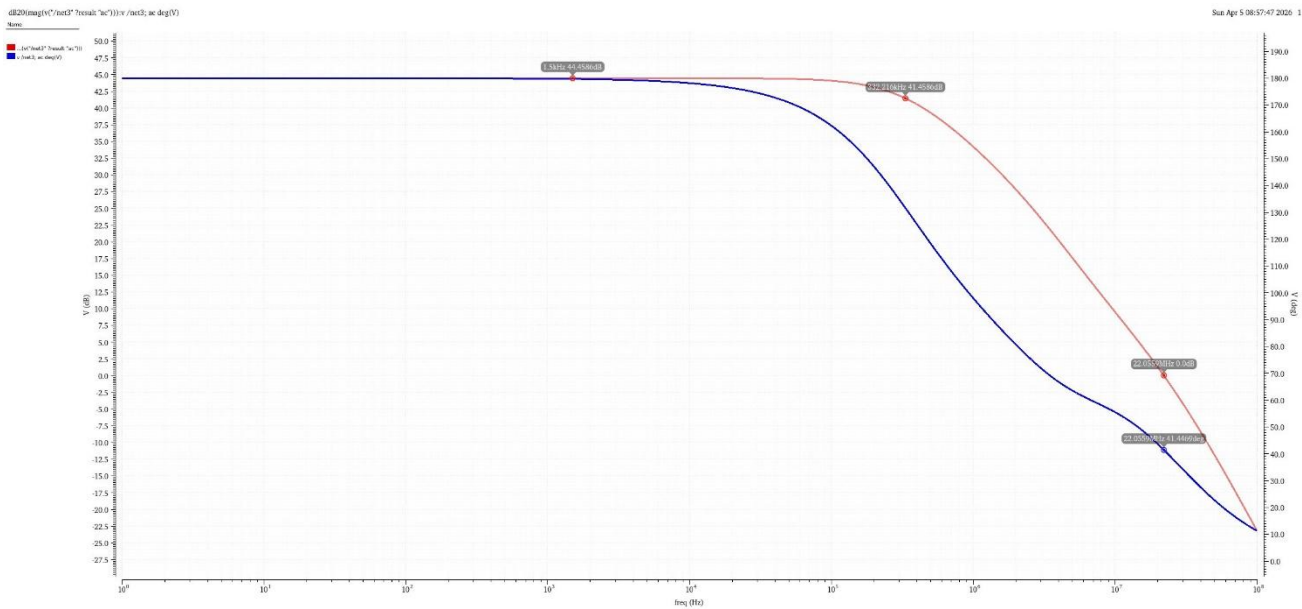


图 4.4.3 OTA 幅频响应和相频响应

4.5 微环调制器 Verilog-A 行为级建模

前级 DAC 虽然在物理层面上采用了 4bit 粗分辨率架构，但在闭环基带内成功实现了极高的模拟重构精度（SNDR 达 90.09dB，有效位数 ENOB 约 14.67bits）。这一接近 15bits 的高保真模拟电压输出，为微环调制器的精细波长调谐与高阶调制奠定了基础。

为了在光电协同仿真环境中准确验证该高精度驱动信号对光路的实际调制效果，系统采用 Verilog-A 语言构建了微环调制器的多物理场行为级模型，完整涵盖了从电学驱动、热学动态响应到光学传输的三个核心转换阶段：

1. 电-热动态转换

微环的热调谐机制首先依赖于施加在微加热器上的驱动电压 V_{heater} ，根据欧姆定律，流过加热器的电流及产生的焦耳热由下式决定：

$$I_{heater} = \frac{V_{heater}}{R_{heater}}$$

由于热传导过程存在一定的延迟与惯性，加热功率 P_{heat} 引起的光波导核心区温度变化 ΔT 并非瞬时响应，而是表现为一阶低通特性：

$$\Delta T(s) = \frac{R_{th}}{1 + s\tau_{th}} \cdot P_{heat}(s)$$

式中， R_{th} 为微环波导的热阻， τ_{th} 为系统的热时间常数。在 Verilog-A 建模中，我们引入 RC 等效低通滤波器网络里实现相应功能，以准确模拟由于热惯性引入的带宽限制。

2. 热光效应与波长漂移机制

硅基波导具有显著的热光效应，折射率的改变将直接导致微环谐振波长的线性漂移。基于微扰理论，温度变化 ΔT 引起的谐振波长偏移量 $\Delta \lambda$ 的计算公式如下：

$$\Delta \lambda = \lambda_{res0} \frac{dn/dT}{n_g} \cdot \Delta T$$

式中， λ_{res0} 为微环的初始谐振波长， dn/dT 为波导材料的热光系数， n_g 为群折射率。由此推导出，波长漂移量与温度变化量成正比，进而与驱动电流的平方成正比：

$$\Delta \lambda \propto \Delta T(s) \propto I_{heat}^2$$

由于前级 DAC 提供了近 15bit 的高分辨率驱动电压，这使得系统能够在极小的电流步进下，通过上述机制对 $\Delta \lambda$ 进行 pm 级别的紧密控制。

3. 光学相位失配与传输衰减模型

微环的最终输出光功率取决于输入激光波长与微环实时谐振波长之间的相位失配。当工作激光波长为 λ_{laser} ，而微环的谐振波长变为 $\lambda_{res} = \lambda_{res0} + \Delta \lambda$ ，波导环路内累积的单圈相位差 ϕ

可表示为：

$$\phi = 2\pi n_g L \left(\frac{1}{\lambda_{laser}} - \frac{1}{\lambda_{res}} \right)$$

式中， L 为微环的周长。基于全通型微环谐振器的稳态干涉理论，其光强透射率 T 为相位的周期函数：

$$T = \frac{a^2 + t^2 - 2at \cos \phi}{1 + a^2 t^2 - 2at \cos \phi}$$

式中， a 为光在微环内部传输一圈的单圈幅度衰减系数， t 为直通波导与微环之间的自耦合系数。

最终，经过微环调制后的输出光功率：

$$P_{out} = P_{in} \cdot T$$

上述公式体系在 Verilog-A 中被转化为连续时间的模拟行为代码，在 Cadence Virtuoso 中进行仿真，得到光强和波长随 DAC 电压线性增大的关系图 4.5.1。如图所示，该微环的波长随电压增加变大，并在 1552.5nm 处发生谐振。该模型成功将前级 DAC 提供的高精度 V_{heater} 映射为非线性的光学透射率,从而闭环了整个数字前端到光子后端的仿真链路，确保了系统级指标评估的准确性。

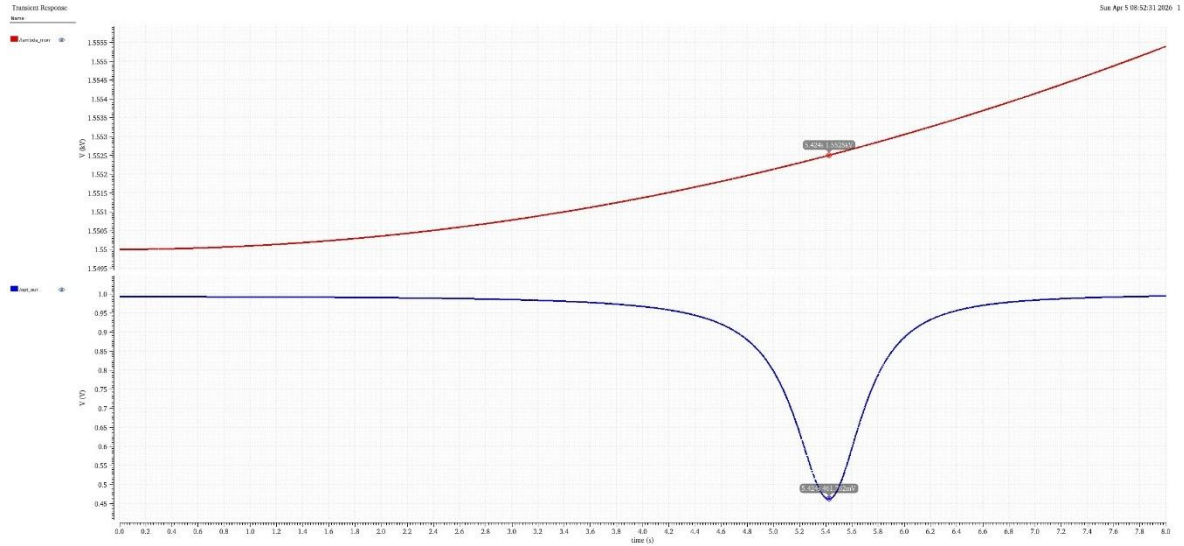


图 4.5.1 光强与波长仿真结果

5 系统集成仿真与结果分析

5.1 插值滤波器与 DSM 调制器级联 FPGA 功能验证

为了验证数字前端逻辑的正确性与系统级噪声整形性能，本项目在 Xilinx Vivado 平台上完成了插值滤波器与 3 阶 4-bit Sigma-Delta 调制器的底层 RTL 级联代码编写，并进行了行为级仿真与 ILA 硬件抓取两次交叉验证。

一、行为级逻辑仿真

在 Vivado 仿真环境下，首先向插值滤波器输入标准的高精度测试基带信号，经滤波器上采样并滤除镜像后，无缝馈入 DSM 模块。

通过导出 DSM 输出的高速 4-bit 码流并进行 FFT 频谱分析计算，得到图 5.1.1。由图可知，该纯数字逻辑在理想状态下的有效位数约为 16.7bits。该结果与前期 MATLAB/Simulink 算法理论仿真的精度高度吻合，证明了 CSD 编码近似以及底层移位加减法逻辑映射的绝对正确性，未引入明显的算法截断误差。

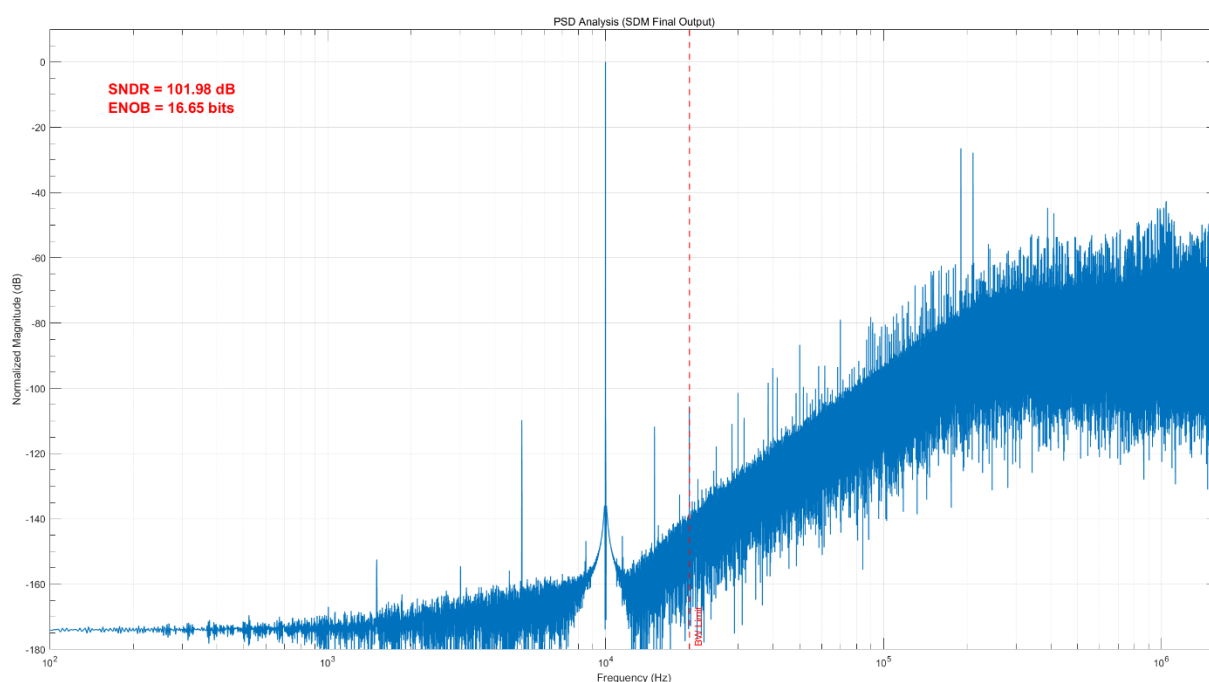


图 5.1.1 行为级仿真频谱图

二、FPGA 功能验证

为检验设计在真实物理环境下的时序约束与运行状态，团队将级联后的数字前端综合为比特流（Bitstream），并下载至目标 FPGA 开发板中运行。利用 Vivado 内置的集成逻辑分析仪(ILA,Integrated Logic Analyzer) 核心，直接在 FPGA 芯片内部捕获时钟驱动

下真实生成的高速 4-bit 码流数据。对捕获的硬件实时数据再次进行频谱提取与测算，得到时域图 5.1.2 和频谱图 5.1.3。通过对实时数据进行频谱分析，测得实际硬件运行的 ENOB 约为 15.98bits。

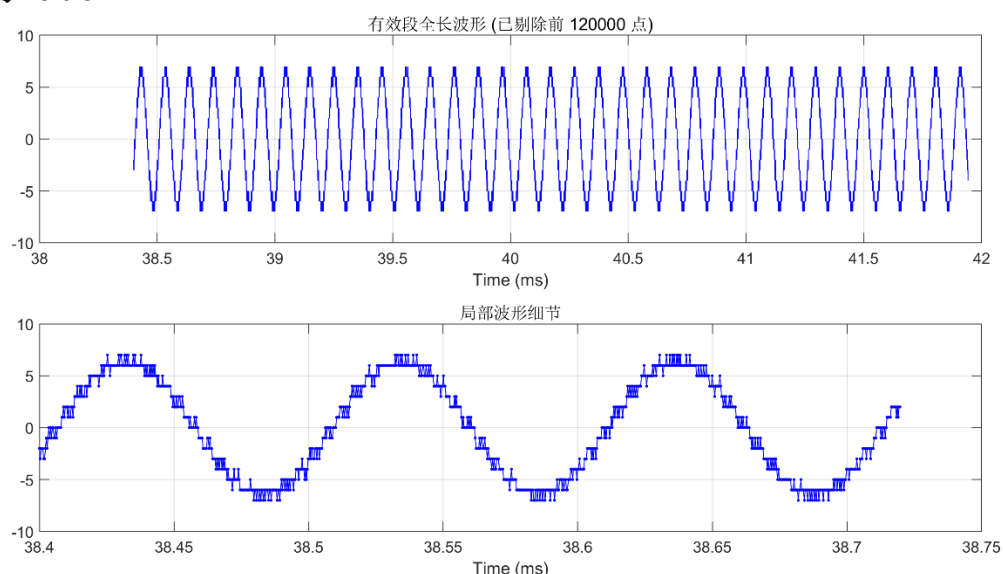


图 5.1.2 功能验证时域图

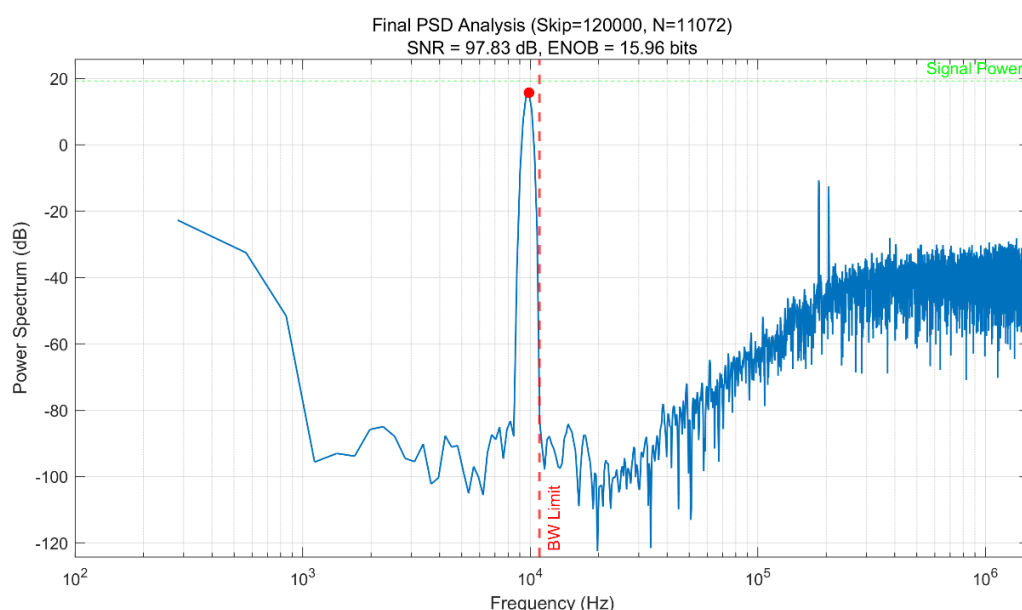


图 5.1.3 功能验证频谱图

对比两次验证结果，实际上板 ILA 抓取数据的有效位数 (15.98 bits) 较行为级仿真 (16.8bits) 出现了约 0.82bits 的微小性能折损。在实际工程中受限物理时钟抖动、电源噪声扰动及 ILA 采样深度限制，该性能损失属于正常现象。该实验结果验证了“高精度插值滤波+数字高阶 DSM”算法的硬件可行性，能够为后级模拟 DAC 与微环提供极其可靠的高精度控制码流。

5.2 系统全链路级联仿真与调节指标验证

为了全面评估高精度 DSM 数模混合驱动架构对微环调制器的实际控制效能，本节搭建了完整的系统级联测试平台。如图 5.2.1 所示，系统级联链路包含了完整的数字与模拟核心模块：前端数字模块（插值滤波器与 Sigma-Delta 调制器）负责生成高保真的 4bit 码流；中间模拟模块（4-bit DAC 与 OTA 功率驱动级）完成电压重构与电流放大；最终输出驱动级联到上一节构建的微环 Verilog-A 行为模型中，实现热调谐闭环仿真。

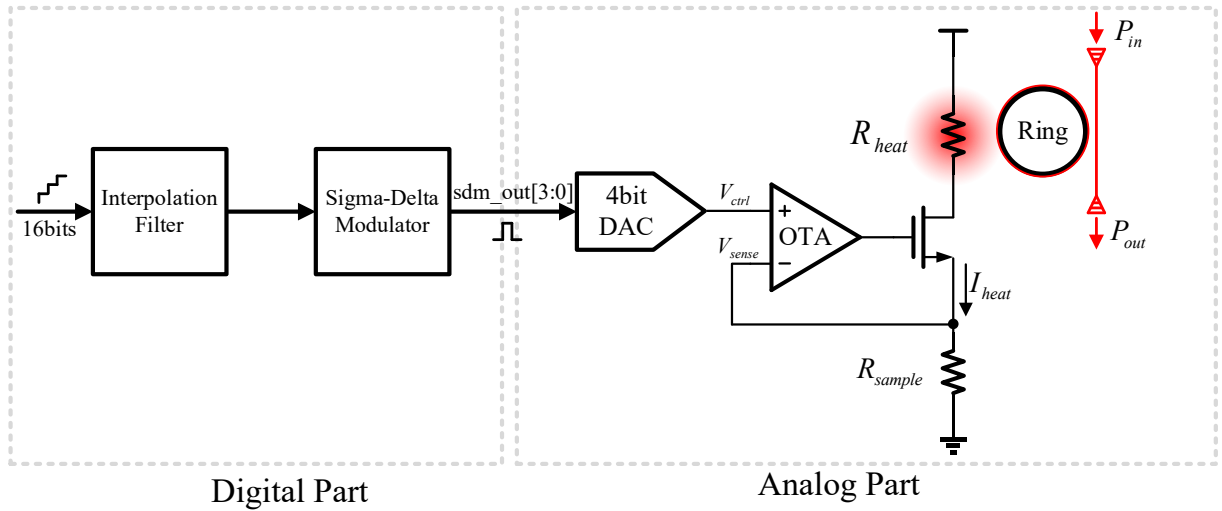


图 5.2.1 测试平台架构图

为了分别验证系统的大范围覆盖能力和极限微调精度，仿真平台分别采用了不同步长和阶数的阶梯输入信号进行测试。

稳态波长调节范围验证

在稳态波长调节范围测试中，系统主要验证 OTA 驱动级与微环加热器在全量程输入下的最大热致相移能力。在数字基带端输入完整的满量程阶梯扫频信号，采用 32 步大步长作为粗调驱动指令，观察微环谐振波长的稳态响应包络。仿真结果如图 5.2.2 所示，从大步长阶梯波形响应图可以看出，随着驱动信号的步进，微环谐振波长呈现出良好的单调变化趋势。系统最终实现了高达 5.3 nm 的总波长调节范围，证明该驱动电路具备提供足够大焦耳热功率的能力。

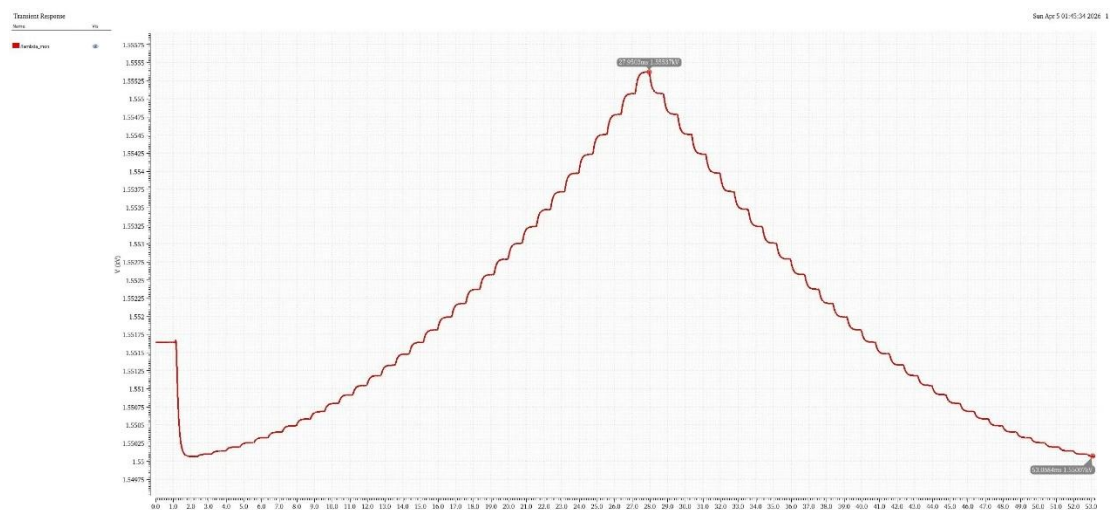


图 5.2.2 稳态波长调节范围仿真图

瞬态波长调节精度验证

对于高阶微环调制系统而言，波长锁定的精度至关重要。该项测试主要验证 DSM 调制器 15bit ENOB 能否有效转化为光学微调精度，在数字端输入局部微小变化的阶梯指令，采用 1024 步小步长进行细调测试，并重点观测每一次阶跃切换时的瞬态建立过程与最终稳态值，得到仿真结果如图 5.2.3 所示，在局部放大的瞬态响应图中，系统能够清晰分辨微小的阶跃变化，在每次小步长切换后，波长响应经过短暂动态建立过程后能迅速稳定。测试表明，系统的极限波长调节精度达到 1.0pm。

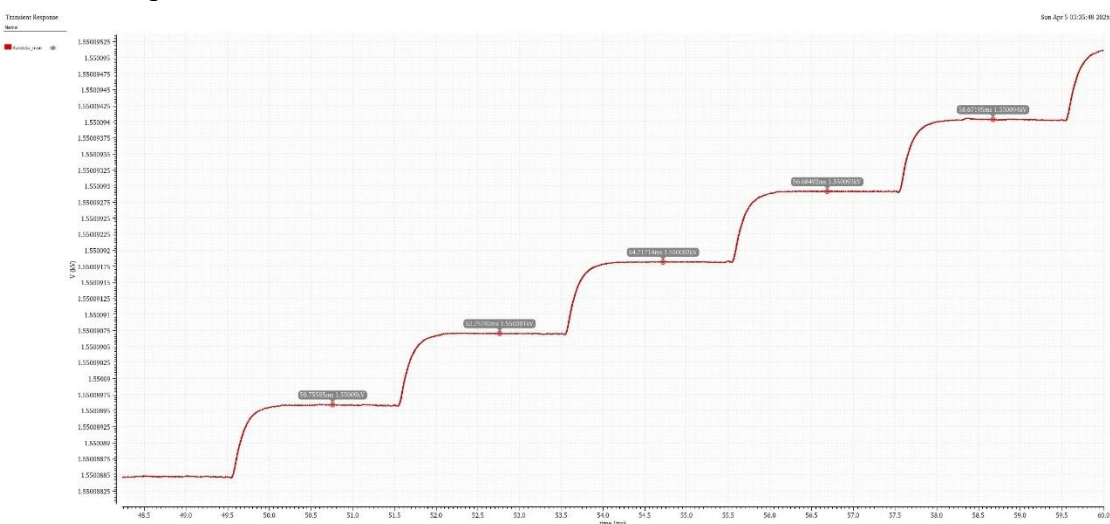


图 5.2.3 瞬态波长调节精度仿真图

6 系统局限性分析与性能优化展望

6.1 系统局限性分析

6.1.1 闭环反馈机制的缺失

目前系统采用的是开环驱动架构，即根据预设的数字控制字直接输出电压。然而，微环调制器作为热敏感器件，其折射率会随环境温度的微小波动而发生显著漂移。由于系统内部缺乏实时的光功率监控反馈环路（如反馈转换电路），驱动器无法感知微环实际工作点与目标波长之间的偏差，难以在环境温度波动的条件下实现长期的波长自动锁定。

6.1.2 映射逻辑非线性

根据微环模型，波长偏移量与加热功率成线性关系，而加热功率与驱动电压之间存在平方关系。在当前的电路架构中，数字输入字与输出电压基本呈线性映射，这意味着数字域的单位变化（1LSB）在不同的电压区间内对应的波长调节步长是不一致的。这种非线性映射关系增加了后端算法控制的复杂度，导致在全量程范围内调节分辨率的有效性受到限制，不利于实现平滑且高线性的高精细调节。

6.2 性能优化展望

6.2.1 数字域平方根压缩电路

针对功率的非线性问题，一种具有前瞻性的思路是利用先进制程中晶体管的物理跨导特性来构建模拟域的平方根压缩电路。相比于在数字域使用 CORDIC 等算法进行补偿，直接利用晶体管在特定偏置下的 I-V 特性来实现“电压-功率”的非线性对冲，能够以更低的硬件开销和功耗实现全链路线性化。

6.2.2 动态元件匹配（DEM）技术

针对电流舵型 DAC 模块，未来可考虑引入随机旋转算法（DWA）等动态元件匹配技术。该技术能通过数字逻辑将电流源阵列间的工艺失配转化为高频带外噪声。在不增加模拟单元面积的前提下，这种方法能有效改善驱动电压的微分非线性（DNL）指标。

7 参考文献

- [1] 牛葳. MEMS 陀螺中 16 位数模转换器设计[D]. 哈尔滨工业大学, 2020. DOI:10.27061/d.cnki.ghgdu.2020.004515.
- [2] 曹志军. 硅基电光调制器的研制[D]. 贵州大学, 2018.
- [3] Li, Hao, et al. "A 25 Gb/s, 4.4 V-swing, AC-coupled ring modulator-based WDM transmitter with wavelength stabilization in 65 nm CMOS." *IEEE Journal of Solid-State Circuits* 50.12 (2015): 3145-3159.
- [4] Kramnik, Danielius, et al. "Scalable feedback stabilization of quantum light sources on a CMOS chip." *Nature Electronics* 8.7 (2025): 620-630.
- [5] Chen, Jianzhong, and Yong Ping Xu. "A novel noise-shaping DAC for multi-bit sigma-delta modulator." *IEEE Transactions on Circuits and Systems II: Express Briefs* 53.5 (2006): 344-348.
- [6] Dallago, Enrico, et al. "Comparison between PWM and sigma-delta modulation in a power factor correction system." *2002 IEEE 33rd Annual IEEE Power Electronics Specialists Conference. Proceedings (Cat. No. 02CH37289)*. Vol. 2. IEEE, 2002.
- [7] Siew, Shawn Yohanes, et al. "Review of silicon photonics technology and platform development." *Journal of Lightwave Technology* 39.13 (2021): 4374-4389.
- [8] Masood, Adil, et al. "Comparison of heater architectures for thermal control of silicon photonic circuits." *10th International Conference on Group IV Photonics*. IEEE, 2013.
- [9] Hawksford, Malcolm J. "SDM versus PWM power digital-toanalogue converters (PDAC) in high-resolution digital audio applications." *Proceedings of the 118th Convention of Audio Engineering Society (AES'05)*. 2005.

8 附件：设计代码说明

8.1 Matlab 代码

插值滤波器系数计算

```
%% ----- 系统采样率配置 -----  
BW      = 10e3;  
OSR     = 128;  
fs_out  = 3.2e6;  
fs_in   = fs_out/OSR;  
L1=2; L2=4; L3=16;  
fs1 = fs_in * L1;  
fs2 = fs1    * L2;  
fprintf('fs_in=%.3f kHz, fs1=%.3f kHz, fs2=%.3f kHz, fs_out=%.3f MHz\n', ...  
        fs_in/1e3,fs1/1e3,fs2/1e3,fs_out/1e6);  
  
%% =====  
%   Stage-1 : 2× Halfband FIR (微调权重以满足 0.000250 限制)  
% =====  
Ap1_dB = 0.00024; % 稍微收紧设计目标，确保余量点  
As1_dB = 100;  
devp1 = (10^(Ap1_dB/20)-1)/(10^(Ap1_dB/20)+1); devs1 = 10^(-As1_dB/20);  
fp1 = 0.44*(fs1/2); fsb1= 0.56*(fs1/2);  
  
% 增加通带权重 w1_weight (1.2)，强制减小通带抖动  
[n1,fo1,ao1,w1_vals] = firpmord([fp1 fsb1]/(fs1/2), [1 0], [devp1 devs1]);  
h1 = firpm(101, fo1, ao1, [1.2 1]);  
h1 = h1/abs(sum(h1)); % 归一化  
  
spec1 = measure_specs_fir(h1,fs1,11.0125e3,13.9875e3,'Stage-1 半带滤波器');  
print_spec_table(spec1,0.000250,90);  
  
%% =====
```

```

% Stage-2 : 4× 低通 FIR
% =====
fp2=11e3; fsb2=38e3; Ap2_dB=0.002; As2_dB=100;
devp2=(10^(Ap2_dB/20)-1)/(10^(Ap2_dB/20)+1); devs2=10^(-As2_dB/20);
h2 = firpm(41, [0 fp2*2/fs2 fsb2*2/fs2 1], [1 1 0 0]);
h2 = h2/abs(sum(h2));

spec2 = measure_specs_fir(h2,fs2,fp2,fsb2,'Stage-2 1/4 带低通');
print_spec_table(spec2,0.002,70);

%% =====
% 级联整体
% =====
Nw = 2^15; w = linspace(0,pi,Nw);
Htot = polyval(h1,exp(1j*w/4).^(-1)) .* polyval(h2,exp(1j*w).^(-1)); fHz = w/(2*pi)*fs2;
pb=fHz<=10e3; sb=fHz>=40e3 & fHz<=fs2/2;
Htot_mag = abs(Htot) / abs(Htot(1));
fprintf("\n----- 级联整体 -----\n");
fprintf('通带(0-10 kHz) 纹波 : %.4f dB\n',20*log10(max(Htot_mag(pb))/min(Htot_mag(pb))));
fprintf('阻带(>=40 kHz) 最大抑制 : %.1f dB\n',-20*log10(min(Htot_mag(sb))));
fprintf('阻带(>=40 kHz) 最差抑制 : %.1f dB\n',-20*log10(max(Htot_mag(sb))));

[~,h1_hex]=quantize_to_q15_hex(h1);
[~,h2_hex]=quantize_to_q15_hex(h2);
writelines(h1_hex,'coeff_hbf_int16.hex'); writelines(h2_hex,'coeff_qbf_int16.hex');
fprintf("\nExported: coeff_hbf_int16.hex, coeff_qbf_int16.hex\n");

function s=ternary(cond,a,b)
    if cond, s=a; else, s=b; end
end

function [hq,hex]=quantize_to_q15_hex(h)
    hq=round(h*32767);
    hex=arrayfun(@(v) sprintf('%04X', typecast(int16(v), 'uint16')), hq, 'UniformOutput', false);

```

end

DSM 调制器参数计算

```
order=3; OSR=128; opt=1; H_inf=1.6; f0=0; xlim=0.9; nlev=16; form='CIFB';
H0 = synthesizNTF(order, OSR, opt, H_inf, f0);
[a, g, b, c] = realizeNTF(H0, form);
b(2:end) = 0;
ABCD0 = stuffABCD(a, g, b, c, form);
[ABCDs, umax] = scaleABCD(ABCD0, nlev, f0, xlim);
[as, gs, bs, cs] = mapABCD(ABCDs, form);
[NTFs, STFs] = calculateTF(ABCD0, 1);
```

8.2 Verilog HDL 代码

半带滤波器

```
module fir_stage1 (
    input  wire          clk,          // 3.2MHz 主时钟
    input  wire          rst_n,
    input  wire          en,          // 使能信号 (由顶层计数器产生, 50kHz
频率)
    input  wire signed [15:0] data_in,
    output reg  signed [15:0] data_out // 修改为 reg, 方便数据保持
);

// 1. 移位寄存器流水线
reg signed [15:0] shift_reg [0:101];
integer i;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (i=0; i<102; i=i+1) shift_reg[i] <= 0;
    end else if (en) begin // 只有在使能有效时才进行移位
        shift_reg[0] <= data_in;
        for (i=1; i<102; i=i+1) shift_reg[i] <= shift_reg[i-1];
    end
end
```

```
    end  
end
```

```
// 2. 对称项预加 (组合逻辑, 随 shift_reg 自动变化)
```

```
wire signed [16:0] pre_sum_0 = shift_reg[0] + shift_reg[101];  
wire signed [16:0] pre_sum_1 = shift_reg[1] + shift_reg[100];  
wire signed [16:0] pre_sum_2 = shift_reg[2] + shift_reg[99];  
wire signed [16:0] pre_sum_3 = shift_reg[3] + shift_reg[98];  
wire signed [16:0] pre_sum_4 = shift_reg[4] + shift_reg[97];  
wire signed [16:0] pre_sum_5 = shift_reg[5] + shift_reg[96];  
wire signed [16:0] pre_sum_6 = shift_reg[6] + shift_reg[95];  
wire signed [16:0] pre_sum_7 = shift_reg[7] + shift_reg[94];  
wire signed [16:0] pre_sum_8 = shift_reg[8] + shift_reg[93];  
wire signed [16:0] pre_sum_9 = shift_reg[9] + shift_reg[92];  
wire signed [16:0] pre_sum_10 = shift_reg[10] + shift_reg[91];  
wire signed [16:0] pre_sum_11 = shift_reg[11] + shift_reg[90];  
wire signed [16:0] pre_sum_12 = shift_reg[12] + shift_reg[89];  
wire signed [16:0] pre_sum_13 = shift_reg[13] + shift_reg[88];  
wire signed [16:0] pre_sum_14 = shift_reg[14] + shift_reg[87];  
wire signed [16:0] pre_sum_15 = shift_reg[15] + shift_reg[86];  
wire signed [16:0] pre_sum_16 = shift_reg[16] + shift_reg[85];  
wire signed [16:0] pre_sum_17 = shift_reg[17] + shift_reg[84];  
wire signed [16:0] pre_sum_18 = shift_reg[18] + shift_reg[83];  
wire signed [16:0] pre_sum_19 = shift_reg[19] + shift_reg[82];  
wire signed [16:0] pre_sum_20 = shift_reg[20] + shift_reg[81];  
wire signed [16:0] pre_sum_21 = shift_reg[21] + shift_reg[80];  
wire signed [16:0] pre_sum_22 = shift_reg[22] + shift_reg[79];  
wire signed [16:0] pre_sum_23 = shift_reg[23] + shift_reg[78];  
wire signed [16:0] pre_sum_24 = shift_reg[24] + shift_reg[77];  
wire signed [16:0] pre_sum_25 = shift_reg[25] + shift_reg[76];  
wire signed [16:0] pre_sum_26 = shift_reg[26] + shift_reg[75];  
wire signed [16:0] pre_sum_27 = shift_reg[27] + shift_reg[74];  
wire signed [16:0] pre_sum_28 = shift_reg[28] + shift_reg[73];  
wire signed [16:0] pre_sum_29 = shift_reg[29] + shift_reg[72];
```

```

wire signed [16:0] pre_sum_30 = shift_reg[30] + shift_reg[71];
wire signed [16:0] pre_sum_31 = shift_reg[31] + shift_reg[70];
wire signed [16:0] pre_sum_32 = shift_reg[32] + shift_reg[69];
wire signed [16:0] pre_sum_33 = shift_reg[33] + shift_reg[68];
wire signed [16:0] pre_sum_34 = shift_reg[34] + shift_reg[67];
wire signed [16:0] pre_sum_35 = shift_reg[35] + shift_reg[66];
wire signed [16:0] pre_sum_36 = shift_reg[36] + shift_reg[65];
wire signed [16:0] pre_sum_37 = shift_reg[37] + shift_reg[64];
wire signed [16:0] pre_sum_38 = shift_reg[38] + shift_reg[63];
wire signed [16:0] pre_sum_39 = shift_reg[39] + shift_reg[62];
wire signed [16:0] pre_sum_40 = shift_reg[40] + shift_reg[61];
wire signed [16:0] pre_sum_41 = shift_reg[41] + shift_reg[60];
wire signed [16:0] pre_sum_42 = shift_reg[42] + shift_reg[59];
wire signed [16:0] pre_sum_43 = shift_reg[43] + shift_reg[58];
wire signed [16:0] pre_sum_44 = shift_reg[44] + shift_reg[57];
wire signed [16:0] pre_sum_45 = shift_reg[45] + shift_reg[56];
wire signed [16:0] pre_sum_46 = shift_reg[46] + shift_reg[55];
wire signed [16:0] pre_sum_47 = shift_reg[47] + shift_reg[54];
wire signed [16:0] pre_sum_48 = shift_reg[48] + shift_reg[53];
wire signed [16:0] pre_sum_49 = shift_reg[49] + shift_reg[52];
wire signed [16:0] pre_sum_50 = shift_reg[50] + shift_reg[51];

```

// 3. CSD 无乘法器计算层 (组合逻辑)

```

wire signed [34:0] prod_0 = -pre_sum_0;
wire signed [34:0] prod_1 = pre_sum_1;
wire signed [34:0] prod_2 = (pre_sum_2 << 2) - pre_sum_2;
wire signed [34:0] prod_3 = -(pre_sum_3 << 2) + pre_sum_3;
wire signed [34:0] prod_4 = -(pre_sum_4 << 2) - pre_sum_4;
wire signed [34:0] prod_5 = (pre_sum_5 << 3) - (pre_sum_5 << 1);
wire signed [34:0] prod_6 = (pre_sum_6 << 3) + (pre_sum_6 << 1);
wire signed [34:0] prod_7 = -(pre_sum_7 << 4) + (pre_sum_7 << 2) + pre_sum_7;
wire signed [34:0] prod_8 = -(pre_sum_8 << 4) - pre_sum_8;
wire signed [34:0] prod_9 = (pre_sum_9 << 4) + (pre_sum_9 << 2) - pre_sum_9;
wire signed [34:0] prod_10 = (pre_sum_10 << 5) - (pre_sum_10 << 2);

```

```

wire signed [34:0] prod_11 = -(pre_sum_11 << 5) + (pre_sum_11 << 1);
wire signed [34:0] prod_12 = -(pre_sum_12 << 6) + (pre_sum_12 << 4) + (pre_sum_12 << 2) +
pre_sum_12;
wire signed [34:0] prod_13 = (pre_sum_13 << 6) - (pre_sum_13 << 4) - pre_sum_13;
wire signed [34:0] prod_14 = (pre_sum_14 << 6);
wire signed [34:0] prod_15 = -(pre_sum_15 << 6) - (pre_sum_15 << 2) - pre_sum_15;
wire signed [34:0] prod_16 = -(pre_sum_16 << 7) + (pre_sum_16 << 5) + (pre_sum_16 << 2) -
pre_sum_16;
wire signed [34:0] prod_17 = (pre_sum_17 << 7) - (pre_sum_17 << 5) + (pre_sum_17 << 2);
wire signed [34:0] prod_18 = (pre_sum_18 << 7) + (pre_sum_18 << 2) - pre_sum_18;
wire signed [34:0] prod_19 = -(pre_sum_19 << 7) - (pre_sum_19 << 4) + (pre_sum_19 << 2) -
pre_sum_19;
wire signed [34:0] prod_20 = -(pre_sum_20 << 8) + (pre_sum_20 << 6) + (pre_sum_20 << 4) -
(pre_sum_20 << 2) - pre_sum_20;
wire signed [34:0] prod_21 = (pre_sum_21 << 8) - (pre_sum_21 << 6) + (pre_sum_21 << 1);
wire signed [34:0] prod_22 = (pre_sum_22 << 8) - (pre_sum_22 << 4) + (pre_sum_22 << 2);
wire signed [34:0] prod_23 = -(pre_sum_23 << 8) - (pre_sum_23 << 3) + (pre_sum_23 << 1);
wire signed [34:0] prod_24 = -(pre_sum_24 << 8) - (pre_sum_24 << 6) - (pre_sum_24 << 2) -
pre_sum_24;
wire signed [34:0] prod_25 = (pre_sum_25 << 9) - (pre_sum_25 << 7) - (pre_sum_25 << 5) -
(pre_sum_25 << 2);
wire signed [34:0] prod_26 = (pre_sum_26 << 9) - (pre_sum_26 << 7) + (pre_sum_26 << 5) +
(pre_sum_26 << 3) + (pre_sum_26 << 1);
wire signed [34:0] prod_27 = -(pre_sum_27 << 9) + (pre_sum_27 << 6) - (pre_sum_27 << 3);
wire signed [34:0] prod_28 = -(pre_sum_28 << 9) - (pre_sum_28 << 5) - (pre_sum_28 << 3);
wire signed [34:0] prod_29 = (pre_sum_29 << 9) + (pre_sum_29 << 6) + (pre_sum_29 << 4) -
pre_sum_29;
wire signed [34:0] prod_30 = (pre_sum_30 << 10) - (pre_sum_30 << 8) - (pre_sum_30 << 6) +
(pre_sum_30 << 2) - pre_sum_30;
wire signed [34:0] prod_31 = -(pre_sum_31 << 10) + (pre_sum_31 << 8) + (pre_sum_31 << 3);
wire signed [34:0] prod_32 = -(pre_sum_32 << 10) + (pre_sum_32 << 7) - (pre_sum_32 << 2);
wire signed [34:0] prod_33 = (pre_sum_33 << 10) - (pre_sum_33 << 6) + (pre_sum_33 << 3) +
pre_sum_33;
wire signed [34:0] prod_34 = (pre_sum_34 << 10) + (pre_sum_34 << 7) - (pre_sum_34 << 4) +

```

```

(pre_sum_34 << 2);
    wire signed [34:0] prod_35 = -(pre_sum_35 << 10) - (pre_sum_35 << 8) + (pre_sum_35 << 6) -
(pre_sum_35 << 4) - (pre_sum_35 << 1);
    wire signed [34:0] prod_36 = -(pre_sum_36 << 11) + (pre_sum_36 << 9) + (pre_sum_36 << 7) -
(pre_sum_36 << 5) - (pre_sum_36 << 2) + pre_sum_36;
    wire signed [34:0] prod_37 = (pre_sum_37 << 11) - (pre_sum_37 << 9) + (pre_sum_37 << 5) +
(pre_sum_37 << 2);
    wire signed [34:0] prod_38 = (pre_sum_38 << 11) - (pre_sum_38 << 8) + (pre_sum_38 << 6) -
(pre_sum_38 << 4) - (pre_sum_38 << 2) - pre_sum_38;
    wire signed [34:0] prod_39 = -(pre_sum_39 << 11) + (pre_sum_39 << 5) - (pre_sum_39 << 1);
    wire signed [34:0] prod_40 = -(pre_sum_40 << 11) - (pre_sum_40 << 8) - (pre_sum_40 << 6) +
(pre_sum_40 << 3) + pre_sum_40;
    wire signed [34:0] prod_41 = (pre_sum_41 << 11) + (pre_sum_41 << 9) + (pre_sum_41 << 6) +
(pre_sum_41 << 3) + pre_sum_41;
    wire signed [34:0] prod_42 = (pre_sum_42 << 12) - (pre_sum_42 << 10) + (pre_sum_42 << 5) -
(pre_sum_42 << 1);
    wire signed [34:0] prod_43 = -(pre_sum_43 << 12) + (pre_sum_43 << 9) + (pre_sum_43 << 5)
+ (pre_sum_43 << 3) + pre_sum_43;
    wire signed [34:0] prod_44 = -(pre_sum_44 << 12) - (pre_sum_44 << 7) - (pre_sum_44 << 5) -
(pre_sum_44 << 2) + pre_sum_44;
    wire signed [34:0] prod_45 = (pre_sum_45 << 12) + (pre_sum_45 << 10) - (pre_sum_45 << 6)
+ (pre_sum_45 << 3) - (pre_sum_45 << 1);
    wire signed [34:0] prod_46 = (pre_sum_46 << 13) - (pre_sum_46 << 11) + (pre_sum_46 << 8) -
(pre_sum_46 << 5) + (pre_sum_46 << 1);
    wire signed [34:0] prod_47 = -(pre_sum_47 << 13) - (pre_sum_47 << 5) + (pre_sum_47 << 2) -
pre_sum_47;
    wire signed [34:0] prod_48 = -(pre_sum_48 << 14) + (pre_sum_48 << 12) + (pre_sum_48 << 9)
+ (pre_sum_48 << 6);
    wire signed [34:0] prod_49 = (pre_sum_49 << 14) + (pre_sum_49 << 12) - (pre_sum_49 << 10)
+ (pre_sum_49 << 7) - (pre_sum_49 << 5) + (pre_sum_49 << 3) - (pre_sum_49 << 1);
    wire signed [34:0] prod_50 = (pre_sum_50 << 16) - (pre_sum_50 << 13) + (pre_sum_50 << 11)
- (pre_sum_50 << 9) + (pre_sum_50 << 7) + (pre_sum_50 << 2) + pre_sum_50;

```

// 4. 最终累加逻辑 (只有在使能有效时才计算并更新输出寄存器)

```

    wire signed [34:0] full_sum =
        prod_0 + prod_1 + prod_2 + prod_3 + prod_4 + prod_5 + prod_6 + prod_7 + prod_8 +
prod_9 +
        prod_10 + prod_11 + prod_12 + prod_13 + prod_14 + prod_15 + prod_16 + prod_17 +
prod_18 + prod_19 +
        prod_20 + prod_21 + prod_22 + prod_23 + prod_24 + prod_25 + prod_26 + prod_27 +
prod_28 + prod_29 +
        prod_30 + prod_31 + prod_32 + prod_33 + prod_34 + prod_35 + prod_36 + prod_37 +
prod_38 + prod_39 +
        prod_40 + prod_41 + prod_42 + prod_43 + prod_44 + prod_45 + prod_46 + prod_47 +
prod_48 + prod_49 +
        prod_50;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            data_out <= 16'd0;
        end else if (en) begin
            // 四舍五入截断逻辑 (从 Q17 还原) 并存入输出寄存器
            data_out <= (full_sum + (1 << 16)) >>> 16;
        end
    end

end
endmodule

```

1/4 带滤波器

```

module fir_stage2 (
    input  wire          clk,          // 3.2MHz 主时钟
    input  wire          rst_n,
    input  wire          en,          // 200kHz 使能信号 (由顶层产生)
    input  wire signed [15:0] data_in,
    output reg signed [15:0] data_out // 修改为寄存器型输出
);

```

// 1. 移位寄存器流水线 (仅在 en 有效时工作)

```

reg signed [15:0] shift_reg [0:41];
integer i;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (i=0; i<42; i=i+1) shift_reg[i] <= 0;
    end else if (en) begin
        shift_reg[0] <= data_in;
        for (i=1; i<42; i=i+1) shift_reg[i] <= shift_reg[i-1];
    end
end
end

```

// 2. 对称项预加 (组合逻辑, 随寄存器自动更新)

```

wire signed [16:0] pre_sum_0 = shift_reg[0] + shift_reg[41];
wire signed [16:0] pre_sum_1 = shift_reg[1] + shift_reg[40];
wire signed [16:0] pre_sum_2 = shift_reg[2] + shift_reg[39];
wire signed [16:0] pre_sum_3 = shift_reg[3] + shift_reg[38];
wire signed [16:0] pre_sum_4 = shift_reg[4] + shift_reg[37];
wire signed [16:0] pre_sum_5 = shift_reg[5] + shift_reg[36];
wire signed [16:0] pre_sum_6 = shift_reg[6] + shift_reg[35];
wire signed [16:0] pre_sum_7 = shift_reg[7] + shift_reg[34];
wire signed [16:0] pre_sum_8 = shift_reg[8] + shift_reg[33];
wire signed [16:0] pre_sum_9 = shift_reg[9] + shift_reg[32];
wire signed [16:0] pre_sum_10 = shift_reg[10] + shift_reg[31];
wire signed [16:0] pre_sum_11 = shift_reg[11] + shift_reg[30];
wire signed [16:0] pre_sum_12 = shift_reg[12] + shift_reg[29];
wire signed [16:0] pre_sum_13 = shift_reg[13] + shift_reg[28];
wire signed [16:0] pre_sum_14 = shift_reg[14] + shift_reg[27];
wire signed [16:0] pre_sum_15 = shift_reg[15] + shift_reg[26];
wire signed [16:0] pre_sum_16 = shift_reg[16] + shift_reg[25];
wire signed [16:0] pre_sum_17 = shift_reg[17] + shift_reg[24];
wire signed [16:0] pre_sum_18 = shift_reg[18] + shift_reg[23];
wire signed [16:0] pre_sum_19 = shift_reg[19] + shift_reg[22];
wire signed [16:0] pre_sum_20 = shift_reg[20] + shift_reg[21];

```

// 3. CSD 无乘法器计算层

wire signed [34:0] prod_0 = (pre_sum_0 << 2);

wire signed [34:0] prod_1 = (pre_sum_1 << 5) - (pre_sum_1 << 3) - pre_sum_1;

wire signed [34:0] prod_2 = (pre_sum_2 << 6) - (pre_sum_2 << 4) + pre_sum_2;

wire signed [34:0] prod_3 = (pre_sum_3 << 6) - (pre_sum_3 << 3);

wire signed [34:0] prod_4 = -(pre_sum_4 << 3);

wire signed [34:0] prod_5 = -(pre_sum_5 << 7) - (pre_sum_5 << 5) - (pre_sum_5 << 3) +
(pre_sum_5 << 1);

wire signed [34:0] prod_6 = -(pre_sum_6 << 9) + (pre_sum_6 << 7) + (pre_sum_6 << 5) -
(pre_sum_6 << 1);

wire signed [34:0] prod_7 = -(pre_sum_7 << 9) + (pre_sum_7 << 7) - (pre_sum_7 << 3) +
(pre_sum_7 << 1);

wire signed [34:0] prod_8 = -(pre_sum_8 << 6) - (pre_sum_8 << 2) - pre_sum_8;

wire signed [34:0] prod_9 = (pre_sum_9 << 9) + (pre_sum_9 << 7) - (pre_sum_9 << 1);

wire signed [34:0] prod_10 = (pre_sum_10 << 11) - (pre_sum_10 << 9) - (pre_sum_10 << 7) -
(pre_sum_10 << 3);

wire signed [34:0] prod_11 = (pre_sum_11 << 11) - (pre_sum_11 << 9) + (pre_sum_11 << 5) -
(pre_sum_11 << 3);

wire signed [34:0] prod_12 = (pre_sum_12 << 9) - (pre_sum_12 << 3) - (pre_sum_12 << 1);

wire signed [34:0] prod_13 = -(pre_sum_13 << 11) + (pre_sum_13 << 8) + (pre_sum_13 << 4);

wire signed [34:0] prod_14 = -(pre_sum_14 << 12) - (pre_sum_14 << 7) - (pre_sum_14 << 5) +
(pre_sum_14 << 3) - pre_sum_14;

wire signed [34:0] prod_15 = -(pre_sum_15 << 12) - (pre_sum_15 << 10) + (pre_sum_15 << 7)
- (pre_sum_15 << 5) - (pre_sum_15 << 3);

wire signed [34:0] prod_16 = -(pre_sum_16 << 11) - (pre_sum_16 << 8) + (pre_sum_16 << 6) +
(pre_sum_16 << 2) - pre_sum_16;

wire signed [34:0] prod_17 = (pre_sum_17 << 12) + (pre_sum_17 << 10) - (pre_sum_17 << 8) -
(pre_sum_17 << 5) + (pre_sum_17 << 3) - pre_sum_17;

wire signed [34:0] prod_18 = (pre_sum_18 << 14) - (pre_sum_18 << 11) + (pre_sum_18 << 9) +
(pre_sum_18 << 6) - (pre_sum_18 << 4);

wire signed [34:0] prod_19 = (pre_sum_19 << 15) - (pre_sum_19 << 13) + (pre_sum_19 << 8)
+ pre_sum_19;

wire signed [34:0] prod_20 = (pre_sum_20 << 15) - (pre_sum_20 << 11) + (pre_sum_20 << 8) +
(pre_sum_20 << 5) + (pre_sum_20 << 3) + pre_sum_20;

// 4. 累加逻辑

```
wire signed [34:0] full_sum =  
    prod_0 + prod_1 + prod_2 + prod_3 + prod_4 + prod_5 + prod_6 + prod_7 + prod_8 +  
prod_9 +  
    prod_10 + prod_11 + prod_12 + prod_13 + prod_14 + prod_15 + prod_16 + prod_17 +  
prod_18 + prod_19 +  
    prod_20;
```

// 输出锁存与四舍五入截断 (仅在 en 有效时更新结果)

```
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        data_out <= 16'd0;  
    end else if (en) begin  
        data_out <= (full_sum + (1 << 16)) >>> 15;  
    end  
end
```

```
endmodule
```

插值滤波器顶层

```
module interpolation_top (  
    input wire clk, // 3.2MHz 主时钟  
    input wire rst_n,  
    input wire signed [15:0] data_in, // 来自外部的 25kHz 原始采样  
    output wire signed [15:0] data_out // 最终 3.2MHz 输出  
);
```

// --- 1. 计数器生成各种采样时刻 (0-127 循环) ---

```
reg [6:0] cnt;  
always @(posedge clk or negedge rst_n) begin  
    if (!rst_n) cnt <= 7'd0;  
    else cnt <= cnt + 7'd1;  
end
```

```

// 使能信号定义
wire en_50k  = (cnt[5:0] == 6'd0);    // 每 64 个 clk 跳一次 (Stage-1 计算时刻)
wire en_200k = (cnt[3:0] == 4'd0);    // 每 16 个 clk 跳一次 (Stage-2 计算时刻)

// --- 2. Stage-1 (2x FIR) 补零逻辑 ---
// 输入 25kHz -> 输出 50kHz.
// 在 50kHz 的节拍下: 第 1 次喂 data_in, 第 2 次喂 0.
reg signed [15:0] s1_input;
always @(*) begin
    if (cnt == 7'd0)      s1_input = data_in; // 有效采样时刻
    else if (cnt == 7'd64) s1_input = 16'd0;  // 插零时刻
    else                  s1_input = 16'd0;
end

wire signed [15:0] s1_out;
fir_stage1 u_stage1 (
    .clk(clk),
    .rst_n(rst_n),
    .en(en_50k),
    .data_in(s1_input),
    .data_out(s1_out)
);

// --- 3. Stage-2 (4x FIR) 补零逻辑 ---
// 输入 50kHz -> 输出 200kHz.
// 在 200kHz 的节拍下: 第 1 次喂 s1_out, 后 3 次喂 0.
reg signed [15:0] s2_input;
always @(*) begin
    // Stage-1 输出只在 cnt=0 和 cnt=64 更新
    if (cnt == 7'd0 || cnt == 7'd64)
        s2_input = s1_out;
    else if (cnt[3:0] == 4'd0)
        s2_input = 16'd0; // 其余 200kHz 节拍补零

```

```

        else
            s2_input = 16'd0;
        end

    wire signed [15:0] s2_out;
    fir_stage2 u_stage2 (
        .clk(clk),
        .rst_n(rst_n),
        .en(en_200k),
        .data_in(s2_input),
        .data_out(s2_out)
    );

    // --- 4. Stage-3 (16x Repeat) 采样保持 ---
    // 输入 200kHz -> 输出 3.2MHz.
    // 每一个 200kHz 产生的值重复输出 16 次
    reg signed [15:0] s3_hold;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) s3_hold <= 16'd0;
        else if (en_200k) s3_hold <= s2_out; // 保持 Stage-2 算完的结果
    end

    assign data_out = s3_hold;

endmodule

DSM 调制器电路
`timescale 1ns / 1ps

module sdm_cifb_3rd_4bit(
    input wire clk,
    input wire rst_n,
    input wire signed [15:0] din_16,
    output wire signed [3:0] dout
);

```

```

wire signed [31:0] din = {{7{din_16[15]}}, din_16, 9'd0};

reg signed [31:0] int1, int2, int3;
wire signed [31:0] v_fb;

assign v_fb = {{7{dout[3]}}, dout, 21'd0};

wire signed [31:0] term_a1_din = (din >>> 7) + (din >>> 9) + (din >>> 11);
wire signed [31:0] term_a1_fb = (v_fb >>> 7) + (v_fb >>> 9) + (v_fb >>> 11);

wire signed [31:0] term_a2 = (v_fb >>> 5) - (v_fb >>> 7) + (v_fb >>> 11);

wire signed [31:0] term_c1 = (int1 >>> 1) - (int1 >>> 3) + (int1 >>> 6) + (int1 >>> 7) -
(int1 >>> 10);

wire signed [31:0] term_c2 = int2 - (int2 >>> 2) + (int2 >>> 4) - (int2 >>> 7);

wire signed [31:0] term_g1 = (int3 >>> 11) + (int3 >>> 17);

wire signed [31:0] term_a3 = (v_fb >>> 4) - (v_fb >>> 6);

wire signed [31:0] term_c3 = (int3 << 4) + (int3 << 2) - (int3 >>> 2);

wire signed [31:0] sum_int1 = term_a1_din - term_a1_fb;
wire signed [31:0] sum_int2 = term_c1 - term_a2 - term_g1;
wire signed [31:0] sum_int3 = term_c2 - term_a3;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        int1 <= 32'd0; int2 <= 32'd0; int3 <= 32'd0;
    end else begin
        int1 <= int1 + sum_int1;
        int2 <= int2 + sum_int2;
    end
end

```

```

        int3 <= int3 + sum_int3;
    end
end

wire signed [31:0] quantizer_in = term_c3;

wire signed [7:0] int_part = quantizer_in[28:21];

assign dout = (int_part > 8'sd7) ? 4'sd7 :
               (int_part < -8'sd8) ? -4'sd8 :
               int_part[3:0];

endmodule

```

8.3 VeilogA 代码

```

// 包含 Verilog-AMS 的学科定义 (electrical 等)
`include "disciplines.vams"
// 包含数学常数 (如 `M_PI)
`include "constants.vams"

// 微环谐振器行为模型 (带监控输出)
// 功能: 模拟热光调谐微环, 输出透射光强, 并提供当前谐振波长和温升的监控信号
module microring_behavioral(
    opt_in, opt_out,          // 光输入 / 输出端口 (电学等效振幅)
    heater_p, heater_n,      // 加热器正 / 负极端口
    lambda_mon, temp_mon      // 监控输出: 当前谐振波长 (nm), 当前温升 (K)
);

// 端口声明为 electrical 类型 (电路仿真中电压代表光场振幅或监控信号)
electrical opt_in, opt_out;
electrical heater_p, heater_n;
electrical lambda_mon, temp_mon;

```

励

耗)

率

转换系数

10us~50us

```
// ===== 光学与几何参数 =====
parameter real lambda_laser = 1552.5e-9; // 输入激光波长 (单位: 米), 固定波长激

parameter real lambda_res0 = 1550e-9; // 室温下(无加热)的微环谐振波长(米)
parameter real ng = 4.2; // 波导群折射率 (无单位)
parameter real L = 20e-6; // 微环周长 (米)
parameter real coupling_k = 0.1; // 耦合系数 (0~1, 直通端耦合强度)
parameter real loss_alpha = 0.99; // 环内单圈振幅透射系数 (<1 表示有损

// ===== 热光效应与热模型参数 =====
parameter real dndT = 1.8e-4; // 热光系数 (1/K), 折射率随温度的变化

parameter real R_th = 3000; // 等效热阻 (K/W), 加热功率到温升的

parameter real tau_th = 120e-6; // 热时间常数 (秒), 热响应速度
parameter real heater_res = 70; // 加热器电阻 (欧姆)

// ===== 监控通道低通滤波参数 =====
// lambda_mon 输出需模拟实际探测器的有限带宽, 添加一阶低通滤波
parameter real tau_lambda_mon = 20e-6; // 监控滤波时间常数 (秒), 典型值

// 内部变量 (实数, 用于计算中间结果)
real p_heat, deltaT; // 加热功率 (瓦), 温升 (开尔文)
real lambda_res, delta_lambda; // 当前谐振波长 (米), 波长偏移量 (米)
real delta_lambda_mon, lambda_mon_avg; // 滤波后的波长偏移量和监控输出值
real phi, transmission; // 单圈相位延迟 (弧度), 直通端光强透射率 (0~1)
real r_sqrt; // sqrt(1 - coupling_k)

// 模拟行为描述 (每个时间点均执行)
analog begin
    // 1) 加热器的电气行为: 强制满足欧姆定律  $V = I * R$ 
    I(heater_p, heater_n) <+ V(heater_p, heater_n) / heater_res;
```

deltaT

波

```
// 2) 计算加热器耗散功率:  $P = V * I$ 
// 功率总是非负的, 若为负则取反 (实际不会发生, 但安全处理)
p_heat = V(heater_p, heater_n) * I(heater_p, heater_n);
if (p_heat < 0) p_heat = -p_heat;

// 3) 热模型: 温升的传递函数为  $T(s)/P(s) = R_{th} / (1 + s * \tau_{th})$ 
// 使用 laplace_nd 实现一阶低通滤波, 输入为 p_heat * R_th, 输出为温升
deltaT = laplace_nd(p_heat * R_th, {1}, {1, tau_th});

// 4) 热光效应: 由温升计算谐振波长的偏移量, 并更新当前谐振波长
// 公式:  $\Delta \lambda = \lambda_0 * (dn/dT) / n_g * \Delta T$ 
delta_lambda = lambda_res0 * (dndT / ng) * deltaT;
lambda_res = lambda_res0 + delta_lambda;

// 5) 监控通道 lambda_mon: 实际探测器响应有限, 对谐振波长变化进行低通滤波
// 一阶滤波, 时间常数 tau_lambda_mon
delta_lambda_mon = laplace_nd(delta_lambda, {1}, {1, tau_lambda_mon});
lambda_mon_avg = lambda_res0 + delta_lambda_mon;

// 6) 计算微环直通端的透射率 (基于当前谐振波长和固定激光波长)
// 单圈相位差:  $\phi = 2 \pi * n_g * L * (1/\lambda_{laser} - 1/\lambda_{res})$ 
phi = 2.0 * 'M_PI' * ng * L * (1.0 / lambda_laser - 1.0 / lambda_res);
r_sqrt = sqrt(1.0 - coupling_k);

// 标准微环直通端光强透射率公式 (振幅平方比):
//  $T = (a^2 + |t|^2 - 2 a |t| \cos\phi) / (1 + a^2 |t|^2 - 2 a |t| \cos\phi)$ 
// 其中 a = loss_alpha, |t| = sqrt(1 - coupling_k)
transmission =
    (loss_alpha*loss_alpha + (1.0 - coupling_k)
     - 2.0*loss_alpha*r_sqrt*cos(phi))
    /
```

```

        (1.0 + loss_alpha*loss_alpha*(1.0 - coupling_k)
        - 2.0*loss_alpha*r_sqrt*cos(phi));

//    数值安全裁剪：透射率应在 [0, 1] 范围内
if (transmission < 0) transmission = 0;
if (transmission > 1) transmission = 1;

// 7) 光输出端口：输出光场振幅 = 输入光场振幅 × 透射率（振幅增益）
//    注意：此处假设输入光场振幅对应电压 V(opt_in)，输出为同相位，忽略相
位变化
V(opt_out) <+ V(opt_in) * transmission;

// 8) 监控输出端口：
//    lambda_mon 输出当前谐振波长（单位：纳米，便于观察），即滤波后的波
长值
V(lambda_mon) <+ lambda_mon_avg * 1e9; // 米 → 纳米
//    temp_mon 输出当前温升（单位：开尔文）
V(temp_mon) <+ deltaT; // 单位：K
end
endmodule

```
